

SAPIENZA
UNIVERSITÀ DI ROMA

Semantic Segmentation for Remote Sensing Images

Facoltà di Ingegneria dell'Informazione, Informatica e Statistica

Corso di Laurea Magistrale in Engineering in Computer Science - Ingegneria Informatica

Candidate

Matteo Campana
ID number 1692599

Thesis Advisor

Prof. Aurelio Uncini

Co-Advisors

Dr. Simone Scardapane
Dr. Valerio Masocci

Academic Year 2021/2022

Thesis not yet defended

Semantic Segmentation for Remote Sensing Images
Master's thesis. Sapienza – University of Rome

© 2022 Matteo Campana. All rights reserved

This thesis has been typeset by LaTeX and the Sapthesis class.

Author's email: campana.1692599@studenti.uniroma1.it

Dedicato a tutte le persone che nonstante gli ostacoli
mi hanno impedito di lasciare questo percorso.

Abstract

The detection of objects in images from high-resolution remote sensing (VHR RSI) consists in determining, automatically or semi-automatically, if an image, aerial or satellite, contains one or more entities belonging to the class of interest and, subsequently, in determining their position within the area under investigation.

However, extracting consistent information in urban environments, characterized by an enormous density of objects classified in a confined space or within the same area of interest, remains a particularly demanding challenge.

The two main critical factors that usually affect the process of achieving the basic goals were, on one side, a high level of variability within the same class combined with low variability between classes and, on the other side, the local weather conditions at the time of image shooting.

Recently, promising results toward this goal were obtained making use of deep convolutional neural networks models, instead of classic techniques such as template matching and pixel classification algorithms.

Herein, starting from the image2image translations model based on deep convolutional neural networks we developed an improved method that makes use of the self-distillation with no labels pre-training technique to extract high accuracy semantic-segmentation masks.

Acknowledgments

I thank Professor Simone Scardapane and Dr. Valerio Masocci for their guidance and helpful comments in the computational setup, and for their suggestions during preparation of this thesis.

I am very grateful to my family and friends, especially my parents. Their belief in me has kept my spirits and motivation high during this long and difficult process.

During this period I was able to learn and grow both professionally and personally to lay the foundations for my future. The memories that I have accumulated during this time will accompany me for the rest of my life and I hope they can give me strength in the future times.

Contents

1 Introduction **1**

2 Machine Learning Background and Modern Techniques **7**
2.1 Multi Layer Perceptrons 7
2.1.1 Multilayer Perceptron 10
2.1.2 Backpropagation 11
2.2 Convolutional Neural Networks 12
2.2.1 Convolution Layer — The Kernel 12
2.2.2 Pooling Layer 14
2.2.3 Classification — Fully Connected Layer (FC Layer) 15
2.3 Variational Autoencoders 17
2.3.1 Autoencoders 17
2.3.2 Variational Autoencoders 19
2.3.3 Mathematical Formulation 20
2.3.4 The Reparameterization Trick 22
2.4 Vector Quantization (VQ-VAE) 24
2.4.1 Quantizing Autoencoders 25
2.4.2 Learning the Codebook 27
2.5 Linear Attention Mechanism (LAM) 29
2.5.1 Definition of Dot-Product Attention 29
2.5.2 Generalization of Dot-Product Attention 30
2.5.3 Linear Attention Mechanism 31
2.6 MAResUNet 33

3 Pre-Training Techniques **35**
3.1 OBoW 41
3.2 Benefits of Data Augmentation 43

4 Proposed Models **45**
4.1 Dataset 47
4.1.1 Data description 47
4.1.2 Data Usage 48
4.2 Training Information 50
4.2.1 Loss Functions 50
4.2.2 Evaluation Metrics 51
4.2.3 Training Details 51

4.3 Results 53
4.3.1 Test Scores 53
4.3.2 Validation Scores 54
4.3.3 Discussion 55
4.3.4 Outputs 57

5 Conclusions and Perspectives **59**

Chapter 1

Introduction

The detection of objects in images from high-resolution remote sensing (VHR RSI) consists in determining, automatically or semi-automatically, if an image, aerial or satellite, contains one or more entities belonging to the class of interest and, subsequently, in determining their position within the area under investigation.

In the classic image classification tasks, the problem can be formulated as follows: "given a query, obtain the labels of all the objects present".

In the field of current data analysis, digital images are treated as three-dimensional matrices of significant dimensions. These huge matrices can contain a sub-vector within each cell that can identify the properties of the pixel, such as the color ranges (RGB, sRBV, Adobe RGB, NTSC, DCi-P3) or other components assigned an operator.

The set of these properties, relating to images, is defined in the literature as *"channels"* of the image.

As previously mentioned, the properties of the individual pixels may not be limited only to the chromatic *"channels"*. However, they may also contain additional metadata relating (for example) to the nomenclature of the individual pixels (such as the object to which the pixel belongs).

For example, given the photo of an airplane, it is possible to add a channel to the image to tell whether the pixel is part of the whole that forms the *"plane"* object or it belongs to those that make up the background.

To do this, various techniques and methods have been developed over the years. Currently, the most commonly used one is semantic segmentation, or image segmentation, which consists in grouping parts of an image that belong to the same class of objects.

It is a form of pixel-level *"prediction"*, as each pixel in an image is classified according to a category.

Image segmentation is a more advanced and challenging process that brings the work to a higher level by trying to accurately identify the exact boundary of the objects in the image.

Turning back to the previous example, we intend to create an algorithm/model for images deriving from remote sensing applications, with the "semantic segmentation" technique, capable of identifying whether each pixel belongs to the airplane or the background.

In practice, the task assigns each pixel of the studied area to a specific identification category previously defined in an arbitrary manner.

Over the past 20 years, several studies and reports have demonstrated the usefulness and effectiveness of this methodology in various applications or contexts, such as monitoring urban areas, land use, environmental protection, and traffic control at a given time.

The classification phase generally focuses on extracting specific categories from the image, such as buildings, linear structures (roads, railways, aqueducts), vehicles, and artificial and natural vegetated surfaces (agricultural crops, woods, meadows, swamps, and peatlands).

However, extracting consistent information in urban environments, characterized by an enormous density of objects classified in a confined space or within the same area of interest, remains a particularly demanding challenge.

The two main critical factors that usually affect the process of achieving the basic goals were, on one side, a high level of variability within the same class combined with low variability between classes and, on the other side, the local weather conditions at the time of image shooting.

With regard to the first point, it is possible to observe that, for the same sample to be classified, the data relating to its characteristics, as they are, in our case, size, shape, or color, may vary to a considerable extent between objects belonging to the same class, while samples of different classes differ a lot in these areas.

Buildings identified by the same class are often built in different materials (concrete, brick, glass) and have different structures. Different shapes and colors characterize the vehicles. Depending on the season and the state, the vegetation can vary in density, extension, and color. On the other hand, semantically different objects classified manually may have similar characteristics (such as concrete roofs, concrete sidewalks, and concrete roads).

As far as the weather conditions are concerned, the problem relates to the possibility that these can affect visibility by occluding parts of the image. Cloudy weather or a large amount of air pollution, mainly present in urban or industrial areas, cause

distortions and artifacts in the reference images, which will then recur, causing problems in the various algorithms implemented, leading to errors of various kinds.

Semantic segmentation, however, while remaining a challenging task compared to other areas of application of computer vision (CV), has made great strides thanks to numerous efforts and significant investments.

It is indeed possible to easily find extensive archives of annotated image data to test one's algorithms in this sector. These datasets are currently in public use and freely accessible, and are also constantly updated and maintained by large companies and/or government agencies.

Dataset	images	task	References
Cityscapes	20K	Semantic segmentation	Daimler AG R&D, TU Darmstadt, MPI Informatics, TU Dresden
ADE20K	20K	Semantic segmentation, image to image transllation	University Alicante/Spain
COCO-stuff	164K	Object detection, instance segmentation, semantic segmentation	Microsoft
PASCAL Context	20K	Semantic segmentation	Standford University

Table 1.1. Most Popular and complete semantic segmentation datasets

Each of the mentioned datasets focuses on a particular aspect of semantic understanding. *Cityscapes* focus on the semantic understanding of urban street scenes. *ADE20K* is focused on scenes and landscapes. The COCO-stuff (Common Objects in *COntext-stuff*) dataset instead focuses on object detection and image captioning. Finally, the *PASCAL* dataset Context (extension of the *PASCAL VOC* 2010 detection challenge) focuses on classifying objects in everyday environments such as offices, apartment interiors, and laboratories.

Unlike the above-mentioned datasets, current efforts to achieve accurate aerial segmentation for the purposes listed above are hampered by the scarcity of organic data collections. This problem is likely due to the intrinsic complexity and high costs of the satellite image acquisition processes in terms of time and money.

Unlike natural images, which can be quickly captured and classified, satellite images require high budgets for acquisition and adequate experience to extract the correct masks.

In different contexts, moreover, specific competence is needed to identify the various objects of interest and assign them to the proper classes.

Since the first appearance of satellite images, a field that began to develop in

the civil field from the 1980*s* onwards, several increasingly sophisticated technologies have been used. One of the most complete and clever survey in which much of the work so far done on these issues has been collected and organized can be found in the work of "Cheng et al."[11], titled *"A survey on object detection in optical remote sensing images"*.

Among the main techniques used, we can mention clustering algorithms such as *k-nn*, *k-means*, or decision-making processes such as *random-forest*. However, with the increase in the precision of instruments and sensors, which have made it possible to collect ever more detailed data as well as larger images, these approaches have revealed their limitations, proving to be less and less adequate. Among the main problems that emerged, complexity and computational load were the most significant.

To obtain a high precision level of analysis, it is not sufficient to analyze an image pixel by pixel but it is necessary to consider the context of each object. Models capable of understanding the individual instances to be identified and the context in which they are found have so far provided the most satisfactory results.

This analysis methodology, indicated by the acronym OBIA (Object-based Image Analysis), identifies two macro-areas of interest, the segmentation of images previously described and the classification of detected objects.

Of the two methodologies, semantic segmentation was the one that aroused the greatest interest in the last decade, not only in the field of Computer Vision but also in other disciplines such as biomedical technologies or the automation sector, with a particular interest in the automotive one.

The development and progress of semantic segmentation techniques have promoted, on the other hand, the rapid advancement of the Computer Vision sector as the progress achieved in a given field was immediately applied to the other fields.

This process was initiated by the computer vision community in 2014 with the use for the first time of convolutional neural networks (CNNs) or fully convolutional neural networks (FCNNs), as reported in the paper by Long et al. [31]. The study highlighted how the change of approach, applied to the semantic segmentation competition and carried out using the PASCAL VOC dataset as a reference, had led to an improvement in performance larger than 20%.

Further improvements were introduced by further research developed in the subsequent years, with the presentation on the study of convolutional neural networks made by the DeepMind laboratories by Chen et al. [8].

A further and significant contribution came in 2015 from the field of biomedical sciences, within which a framework was developed for the recognition and analysis of various biomedical data to help in the diagnosis of various diseases.

Specific attention was paid in particular to deformations of the cranial cortex

and the identification of malignant masses that can easily escape, due to their small size, the first check carried out by the human eye.

This framework took the name of UNet from the work of Ronnebbeger et al. [35], in which the encoder-decoder paradigm was introduced, and makes use of two CNNs. The first, with encoder functions, takes care of gradually reducing the representation space through a convolutional filter. The second, with decoder functions, takes the result of the first network as input and performs transpose convolution operations intending to reconstruct the original image, or the segmentation mask given by the target, through various upsampling steps.

All the methodologies reported so far fall within the scope of techniques called supervised learning, through which, given the starting images and target images, i.e., processed images, they attempt to extract the segmentation mask for the samples not yet processed.

In recent years, various techniques and frameworks have emerged that allow us to train the networks without the need to resort to reference images.

These techniques do not use or make limited use of images already processed manually by experts in the field. This makes it possible to achieve excellent results even in the absence of large datasets with adequate availability of already labeled material, thus mitigating the criticalities due to the reduced availability of already labeled material.

The goal of these new methodologies is to generate a reduced-size representation of the features of the image to be analyzed and then use that exact latent representation in a new network that aims to generate the segmentation mask. Excellent results have been reported in the literature in the following models SimCLR (A Simple Framework for Contrastive Learning of Visual Representations, [9]), MoCo (*"Momentum Contrast for Unsupervised Visual Representation Learning"*, He et al. [17]), SwaV (*"Swapping Assignments between Views"*, Caron et al. [5]), SimSiam (*"Exploring Simple Siamese Representation Learning"*, Chen at al. [10]), DINO (*"Emerging Properties in Self-Supervised Vision Transformers"*, Caron et al. [6]), EsViT (*"Efficient Self-supervised Vision Transformers for Representation Learning"*, Li et al. [24]).

Herein, we illustrate a new methodology to extract segmentation masks from images obtained from satellite telemetry systems.

Based on the technique developed in the study *"Mare: Self-supervised multi-attention resu-net for semantic segmentation in remote sensing"* (Masocci et al. [33], 2021), in combination with vector quantization, and the new self-distillation pre-training method DINO [6], in our work, we have tried to improve the model accuracy.

The work covered in this thesis based on MARE hinges on two major focal points. The first concerns a CNN that uses a Residual Network (ResNet) described in the article *"Deep residual learning for image recognition"* (Kaiming He, Xiangyu Zhang,

Shaoqing Ren, and Jian Sun, 2015) as a backbone, combined with an architecture that follows UNet [29] CNN based on the popular FPN framework (*"Feature Pyramid Network"*, Kirillov et al. [23]).

The second lever, which is the peculiarity of the network used in the original paper MaResUNet by the authors Li et al., [25], consists in the insertion of a "Linear Attention Mechanism" between the skip connections within the architecture itself.

The introduction of a "Linear Attention Mechanism" not only allows to effectively select the "features", as demonstrated by the manifold works carried out on these mechanisms but requires a lower computational cost than the most widespread attention algorithms based on "dot-product attention" techniques (Li et al. [25]).

The proof and the computation of the computational complexity will be illustrated in the following chapter "Linear Attention Mechanism".

This thesis aims to study how to implement a "quantization layer" built on a "Codebook", as described in "Neural discrete representation learning" (Oord et al., [38]). The layer which is interposed between encoder and decoder can bring an increase in the accuracy of the tested model, compared to those models that are not devoid of such a layer.

Overall, the main issues addressed in the thesis will be presented in the following order. First, we will deal with the current state-of-the-art background on convolutional neural networks and how they came from classification tasks to image synthesis algorithms. A section will then be presented that deals with the main relevant studies providing the basis of the reported approach. Subsequently, the core section of the thesis will be exposed together with the modus operandi of the various experiments carried out, with the relevant presentation of the results obtained. Finally, in the concluding chapter, a discussion will be provided as to how the model reacted as well as its strengths and weaknesses.

Chapter 2

Machine Learning Background and Modern Techniques

2.1 Multi Layer Perceptrons

In the early 1940's Warren McCulloch, a neuro-physiologist, teamed up with logician Walter Pitts to create a model of how brains work. A simple linear model produced a positive or negative output, given a set of inputs and weights.

This model of computation was intentionally called a neuron because it tried to mimic how the core building block of the brain worked. Just like brain neurons receive electrical signals, McCulloch and Pitts'Neuron received inputs and, if these signals were strong enough, passed them on to other neurons.

However, this model had a problem. It could not learn like the brain. The only way to get the desired output was to set the weights beforehand.

It was only a decade later that Frank Rosenblatt extended this model and created an algorithm that could learn the weights to generate an output. Building onto McCulloch and Pitt's neuron, Rosenblatt developed the Perceptron.

The Perceptron, that neural network whose name evokes how the future looked from the perspective of the 1950s, is a simple algorithm intended to perform binary classification; i.e., it predicts whether input belongs to a specific category of interest or not.

The Perceptron holds a special place in neural networks and artificial intelligence history. The initial hype regarding its performance led to a rebuttal by Minsky and Papert and broader spread backlash that cast a pall on neural network research for decades, a neural net winter that wholly thawed only with Geoff Hinton's research in the 2000s, the results of which have since swept the machine-learning community.

Frank Rosenblatt, the godfather of the Perceptron, popularized it as a device rather than an algorithm. The Perceptron first entered the world as hardware. Rosenblatt, a psychologist who studied and later lectured at Cornell University,

received funding from the U.S. Office of Naval Research to build a machine that could learn. His machine, the Mark I Perceptron, looked like this:

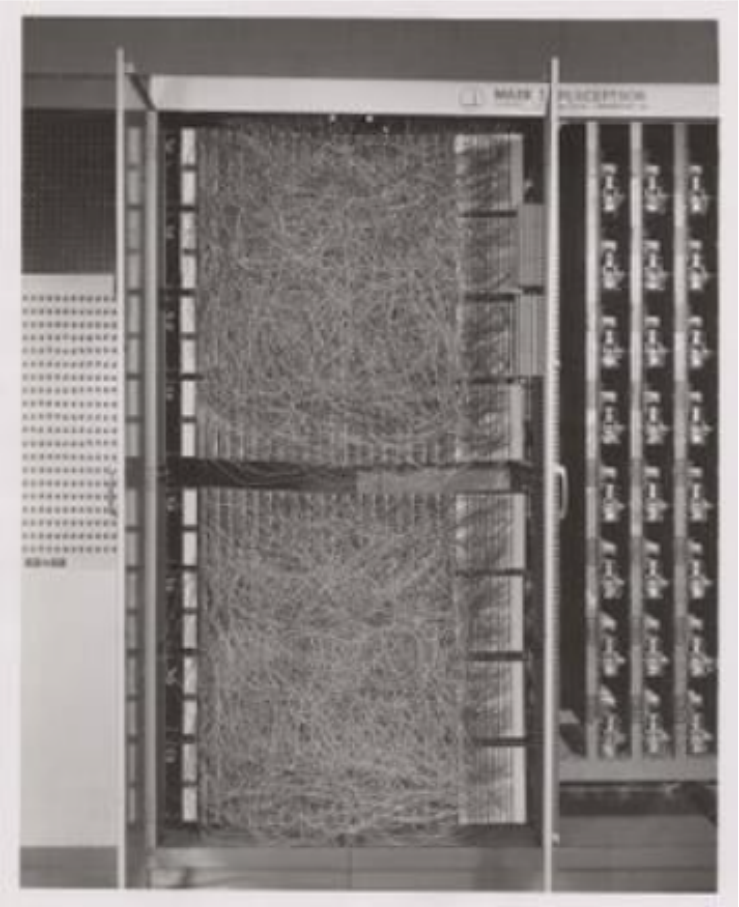


Figure 2.1. Mark I Perceptron machine, the first implementation of the perceptron algorithm. It was connected to a camera with 20×20 cadmium sulfide photocells to make a 400-pixel image. The main visible feature is a patch panel that set different combinations of input features. To the right, arrays of potentiometers that implemented the adaptive weights.[4]

A Perceptron is a linear classifier; it is an algorithm that classifies input by separating categories defining a linear decision boundary. It finds the separating hyperplanes that minimize the distance between misclassified points and the decision boundary. Input is typically a feature vector x multiplied by weights w and added to a bias $b : y = w * x + b$.

A Perceptron produces a single output based on several real-valued inputs by forming a linear combination using its input weights (and sometimes passing the output through a non-linear activation function). The equation below shows how we can formalize the algorithm:

$$y = \sigma(\sum_{i=1}^{n} w_i x_i + b) = \sigma(\mathbf{w}^T \mathbf{x} + \mathbf{b})$$

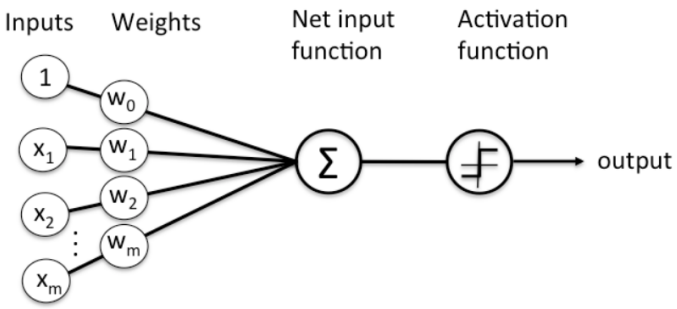


Figure 2.2. Single layer Perceptron graphical representation where $x = [x_1, x_2, \ldots, x_m]^T$ refers to the input vectors, $w = [w_0, w_2, \ldots, w_m]^T$ denote the weights, and Σ the activation function [13]

Where $\mathbf{w}$ denotes the vector of weights, $\mathbf{x}$ is the vector of inputs, $\mathbf{b}$ is the bias and σ is the non-linear activation function, that determines the threshold for the activation of the Perceptron.

To minimize this distance between misclassified points, Perceptron uses Stochastic Gradient Descent as the optimization function. The function determines if the neuron will fire or not. The sigmoid function maps any real input to a value that is either 0 or 1 and encodes a non-linear function. The neuron can receive negative numbers as input, and it will still be able to produce an output that is either 0 or 1.

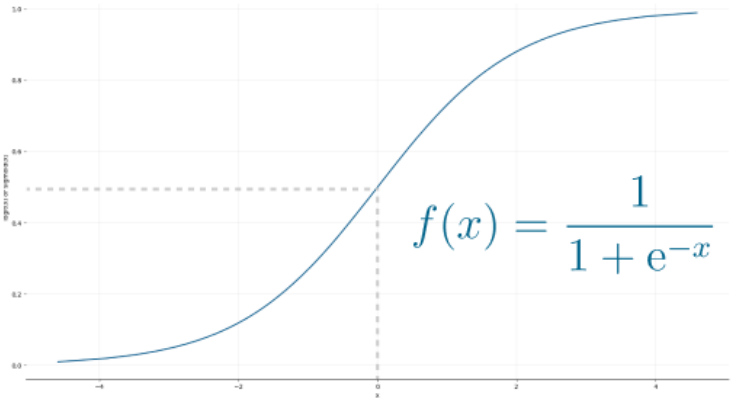


Figure 2.3. Sigmoid function. [3]

Rosenblatt built a single-layer Perceptron. His hardware algorithm did not include multiple layers, which allow neural networks to model a feature hierarchy. Therefore, its hardware circuit was a rudimentary neural network that allows Perceptron to reproduce non-linear functions or algorithm such the XOR operator (XOR operator trigger when input exhibits either one trait or another, but not both; it

stands for "exclusive OR").

Subsequent work with multilayer Perceptrons has shown that they are capable of approximating an XOR operator and many other non-linear functions.

Just as Rosenblatt based the Perceptron on a McCulloch-Pitts neuron, conceived in 1943, so too, Perceptrons themselves are building blocks that only prove to be helpful in such more extensive functions as multilayer Perceptrons.

2.1.1 Multilayer Perceptron

A multilayer Perceptron (MLP) is the most straightforward deep artificial neural network nowadays. They are composed of an input layer to receive the signal, an output layer that makes a decision or prediction about the input, and in between those two, an arbitrary number of hidden layers are the actual computational engine of the MLP. It has been proven that MLPs with one hidden layer can approximate any continuous function.

Multilayer Perceptron falls under the category of feed-forward algorithms because inputs are combined with the initial weights in a weighted sum and subjected to the activation function, just like in the Perceptron. Nevertheless, the difference is that each linear combination is propagated to the next layer.

Multilayer Perceptrons are often applied to supervised learning problems: they train on input-output pairs and learn to model the correlation (or dependencies) between those inputs and outputs. Training involves adjusting the parameters, or the weights and biases, of the model in order to minimize error.

If the algorithm only computed the weighted sums in each neuron, propagated results to the output layer, and stopped there, it would not be able to learn the weights that minimize the cost function. If the algorithm only computed one iteration, there would be no actual learning. This is where **backpropagation** comes into play.

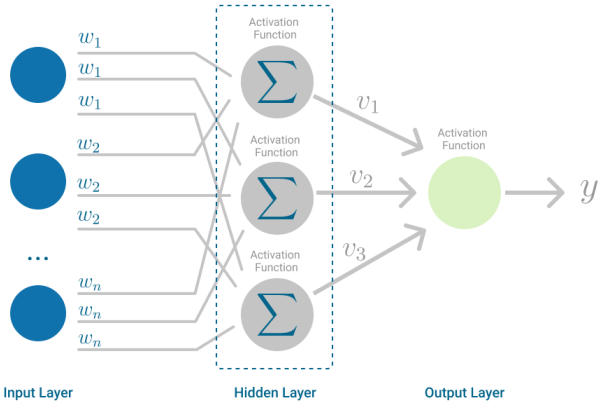


Figure 2.4. Multilayer Perceptron. [3]

2.1.2 Backpropagation

Backpropagation is the learning mechanism that allows the Multilayer Perceptron to iteratively adjust the weights in the network to minimize the cost function.

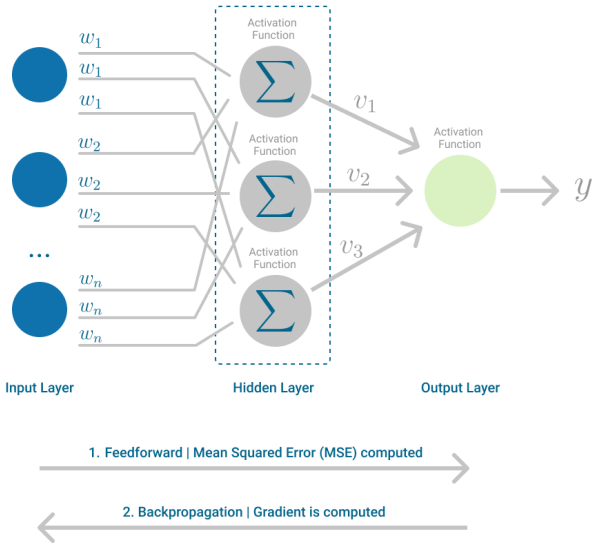


Figure 2.5. Multilayer Perceptron, highlighting the Feedfoward and Backpropagation steps.[3]

In the forward pass, the signal flow moves from the input layer through the hidden layers to the output layer, and the decision of the output layer is measured against the ground truth labels.

$$\Delta_w(t) = -\epsilon \frac{\partial E}{\partial w_{(t)}} + \alpha \Delta_{w_{t-1}}$$

where α represents the learning rate (how wide the convergence step is), ϵ is the bias, E the error, and $\Delta_{w_{t-1}}$ the previous memory state of the weights.

In the backward pass, using backpropagation and the chain rule of calculus, partial derivatives of the error function *w.r.t.* the various weights and biases are back-propagated through the MLP. That act of differentiation gives us a gradient, or a landscape of error, along which the parameters may be adjusted as they move the MLP one step closer to the error minimum. This can be done with any gradient-based optimization algorithm, such as stochastic gradient descent. The network keeps playing that game of tennis until the error falls within a range of tolerance. This state is known as convergence.

2.2 Convolutional Neural Networks

The convolutional neural network (CNN, or ConvNet) is a class of artificial neural networks that have become dominant in various computer vision tasks. CNNs are designed to automatically and adaptively learn spatial hierarchies of features. This procedure is performed by training the model through the backpropagation operation that updates the network's weights with the loss value defined by comparing the current network prediction with the target class. During each pass of the backpropagation algorithm, the network learns to assign importance (learnable weights and biases) to various aspects/objects of the given inputs.

Architectures of ConvNets possess analogous connectivity to the pattern of neurons in the human brain and take inspiration from the organization of the Visual Cortex. Individual neurons respond to stimuli only in a restricted visual field region known as the Receptive Field. A collection of such fields overlap to cover the entire visual area.

ConvNet architectures explicitly assume that the inputs are images, allowing us to encode the architecture's characteristic properties. These then make the forward function more efficient to implement and, lastly, thanks to the convolutional filters, decrease the number of parameters in the network.

ConvNet architectures assume that the inputs are images; an image is nothing but a matrix of pixel values, which allows us to encode specific properties into the architecture. This approach has been shown to preforms much better than its parent, the multi-layer perceptron since it not only learns to reduce the error but can capture the spatial and temporal dependencies in an image through the application of relevant filters.

The given architecture serves a better fitting to the image due to the reduction in the number of parameters involved and re-usability of weights. In other words, we can train the network to understand the image better.

For small images, the multi-layer perceptron is still an option, but when we start to work with big images like dimensions of 8K (7680×4320), the problem will result computationally intensive. The critical component regarding ConvNet is the ability to encode the images into a latent space that is more manageable to process without losing features that are critical for getting a good prediction.

2.2.1 Convolution Layer — The Kernel

Convolution layers preserve the relationship between pixels by learning image features using small windows of input data. This mathematical operator takes two arguments: an image matrix and a filter or kernel.

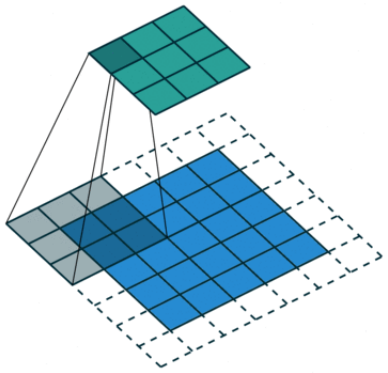


Figure 2.6. Convolution Operation

In the above demonstration, the blue section resembles our $5 \times 5 \times 1$ input image, I. The element involved in carrying out the convolution operation in the first part of a Convolutional Layer is the Kernel/Filter, K, represented in the color green. We have selected K as a $3 \times 3 \times 1$ matrix.

The Kernel steps are defined by the stride value, which defines the breadth space over which the kernel moves along the input image.

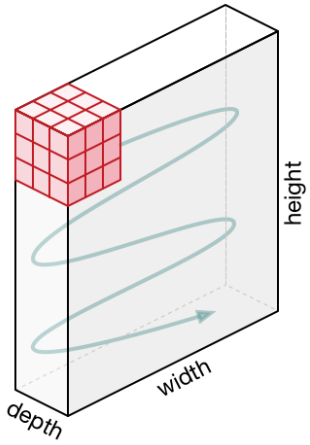


Figure 2.7. Movement of the Kernel

The filter moves to the right with the given *stride* value till it parses the complete width. It hops down to the beginning (left) of the image with the same stride value and repeats the process until it traverses the entire image.

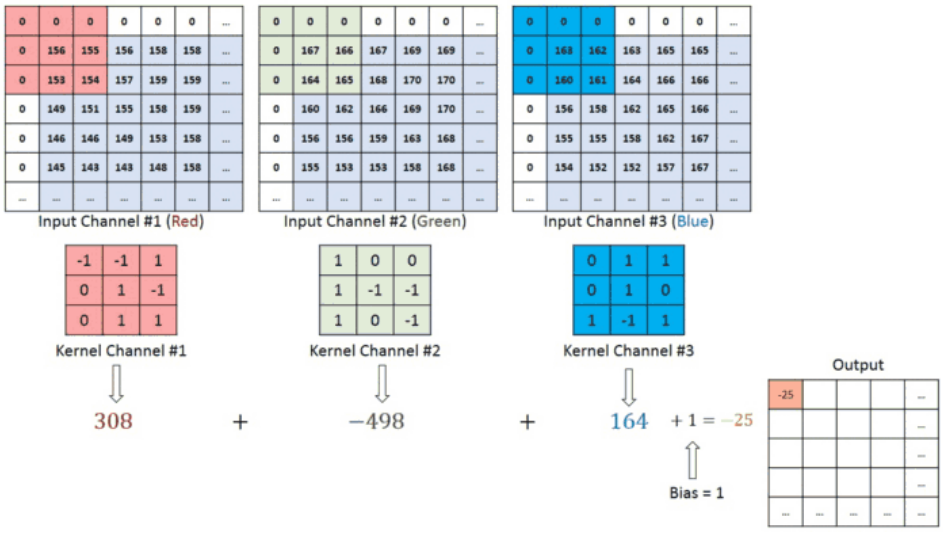


Figure 2.8. Convolution operation on a $M \times N \times 3$ image matrix with a $3 \times 3 \times 3$ Kernel

In the case of images with multiple channels (e.g., RGB), the kernel has the same depth as that of the input image channels. Matrix multiplication is performed between K_n and In stack ($[K_1, I_1]$; $[K_2, I_2]$; $[K_3, I_3]$) and all the results are summed with the bias to give us a squeezed one-depth channel convoluted feature output. Convolution of an image with different filters can perform operations such as edge detection, blur, and sharpening by applying filters.

The objective of the convolution operation is to extract the high-level features, such as edges, from the input image. ConvNets need not be limited to only one Convolutional Layer. Conventionally, the first ConvLayer is responsible for capturing the low-level features such as edges, color, and gradient orientation. With added layers, the architecture adapts to the High-Level features as well, giving us a network that has a wholesome understanding of images

There are two types of results to the operation — one in which the operation reduces in dimensionality the convolved feature compared to the input, and the other in which the dimensionality is either increased or remains the same. We can achieve this result by applying Valid Padding in the case of the former or the same padding in the case of the latter.

2.2.2 Pooling Layer

Similar to the Convolutional Layer, the Pooling layer is responsible for reducing the spatial size of the Convolved Feature. It decreases the computational power required to process the data through dimensionality reduction. Furthermore, it helps extract dominant features, and rotational and positional invariants, thus maintaining the process of effectively training the model.

There are two types of Pooling: Max Pooling and Average Pooling. Max Pooling returns the maximum value from the image portion covered by the kernel. On

the other hand, Average Pooling returns the average of all the values from the image portion covered by the kernel.

Max Pooling also performs as a Noise Suppressant. It discards the noisy activation altogether and performs de-noising and dimensionality reduction. On the other hand, Average Pooling performs dimensionality reduction as a noise suppressing mechanism. Hence, we can say that Max Pooling performs better than Average Pooling.

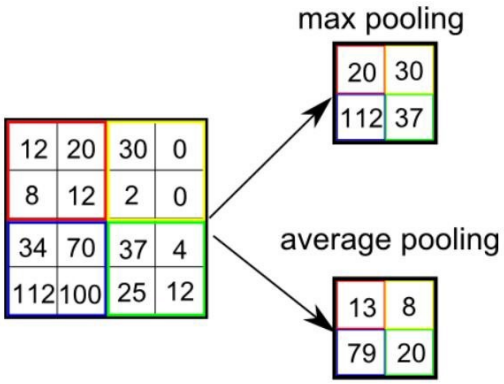


Figure 2.9. Types of Pooling

The Convolutional Layer and the Pooling Layer form the i-th layer of a Convolutional Neural Network. Depending on the complexities and size of the images, the number of such layers may be increased. Deeper neural networks can capture low-level details even better than the small ones, but this comes at the cost of additionally computational power due to the growth of the network's parameters.

After going through the above procedure, we have successfully allowed the model to understand the features. We will flatten the final output and feed it to a regular Neural Network for classification purposes.

2.2.3 Classification — Fully Connected Layer (FC Layer)

In the layer we call as FC layer, we flattened our matrix into a vector and fed it into a fully connected layer. Adding a Fully-Connected layer is the way to match the results of the convolutional operations to fit with the target class dimensions. The fully connected layer is learning to map the input into a probabilistic distribution of our desired target of interest classes. Extracted the distribution, we pass the prediction vector to a softmax function, and sequentially we can perform an argmax operation to see which class obtains the best scores.

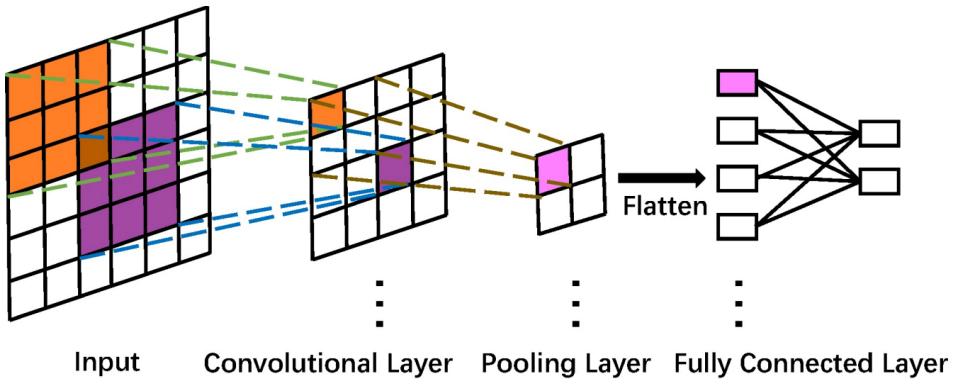


Figure 2.10. A CNN architecture composed of one convolutional layer, one pooling layer and one fully connected layer. The convolutional layer utilizes a 3×3 kernel with stride 1, while the pooling layer utilizes a 2×2 kernel with stride 2. Flatten means converting the feature matrices into vectors.[32]

2.3 Variational Autoencoders

If we want to apply machine learning, we need data, and often the more, the better. To build a classifier like could be a cat detector, labeled images of cats are needed. If we want to find fraudulent credit card transactions with an algorithm, we need to label which transactions are fraudulent and which are not. Nevertheless, what do we do when we have a vast collection of data and no labels.

We might have many unlabeled data, so maybe we have many credit card transactions, but we do not know which ones are fraud and which are not. It would be great if there could be a method somehow to leverage this unlabelled data to learn something interesting, like finding the characteristics that can group the similar ones.

The underlying process of the data could be much more straightforward in a much lower dimensional space than the one in which actual data lie. So many techniques in machine learning try to compress the dimensionality of our data into a smaller space. One prevalent technique used for this purpose is called Variational Autoencoders.

So before we dive into the mechanics of variational-autoencoders, we first want to introduce standard autoencoders.

2.3.1 Autoencoders

Autoencoders are unsupervised learning processes that take advantage of unlabeled data and learn a latent representation of their structure.

That is often useful in many contexts, since it is possible to build a better classifier using autoencoders as a feature extractor. Alternatively, if we do not have any labels, we can use the autoencoders to flag anomalies that might get out labeling process started. It is even possible to use them as a missing values filler or generate new images through synthesis.

Relying on the massive amount of data, well-designed network architectures, and intelligent training techniques, generative models based on auto-encoders have shown an incredible ability to produce highly realistic content of various kinds, such as images, texts, and sounds.

In its simplest form, an autoencoder is a neural network that attempts to do two things: first, it compresses its input data into a lower dimension through the encoder component. The encoder can be simply a group of layers or some more sophisticated model. The autoencoder tries to use this lower-dimensional data representation, defined as the outputs of the encoder and also named the bottleneck of the model, to recreate the original input (see **[Fig. 2.11]**).

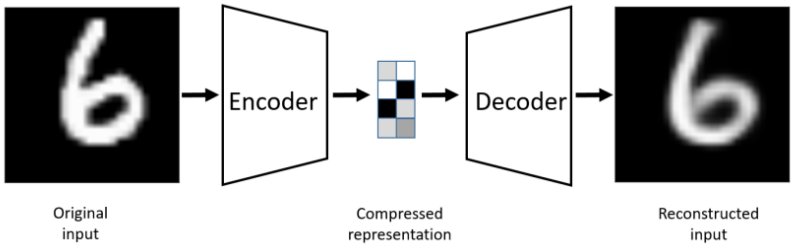


Figure 2.11. An autoencoder example. The input image is encoded to a compressed representation and then decoded. [2]

The *reconstruction error* is the difference between the attempted recreation and the original input. By training to minimize this *reconstruction error*, the network learns to exploit the data's natural structure to find an efficient lower-dimensional representation.

Of course, depending on the initial data distribution, the latent space dimension, and the encoder definition, this compression can be lossy, meaning that a part of the information is lost during the encoding process and cannot be recovered when decoding.

To summarize, if we train a deep convolutional network to do encoding and decoding of many data, we are creating a new kind of compression/dimensionality-reduction algorithm.

A practical example of using this method in the real world is using these networks to reduce the amount of bandwidth we use on our phones. So if we download an image, then the full resolution image is first down-scaled. It is sent through us over the wireless internet connection, and then on our phone, there is a decoder that reconstructs the full resolution image from the compressed representation.

It is interesting to note that the same technique is also applied to image segmentation, so we take an input image and run it through our convolutional encoder. It goes through a bottleneck representation and then remaps to a full output image. However, instead of reconstructing the original image, we are trying to target a segmented version of our input. This type of network is used in self-driving cars to segment the different parts of the public road into specific objects that a car needs to detect.

Imagine that we start with a stock photo; it is a clean image but then we add noise to it, and we will run that noisy image through our encoder network to get through the bottleneck representation. Then we try to reconstruct the image.

Instead of reconstructing the noisy image, we will try and reconstruct the original image. If we train this network with a bunch of these noisy and clean digits, we will try and force the encoder to remove the noise reconstruction the original image; this called a denoising auto-encoder.

Denoising images are not the only thing we can do with this approach. For example, we take input, and instead of adding noise, we crop a rectangular area out of the image. We throw it away and replace it with white or black pixels. We feed that input image through the network, and we try to reconstruct the original full image. This technique is what we call neural-impeding; it is where we take a small part of the image, throw it away, and then we ask the network to reconstruct whatever was there in the input image. With this approach, we can do simple things like removing watermarks from images, but we could also remove a parked car, for example, if we are filming on a movie set in a natural setting.

We can now start to formalize our discussion a little more. The primary purpose of dimensionality reduction techniques is to find the best encoder/decoder pair among a given family. For a given set of possible encoders and decoders, we are looking for the pair that keeps the maximum of information when encoding and, so, has the minimum of *reconstruction error* when decoding. If we denote respectively E and D the families of encoders and decoders we are considering, then the dimensionality reduction problem can be written:

$$(e^*, d^*) = \underset{(e,d)\in E\times D}{argmin}\ \epsilon(x, d(e(x)))$$

where

$$\epsilon(x, d(e(x))$$

defines the *reconstruction error* measure between the input data x and the encoded-decoded data $d(e(x))$. Notice finally that in the following we will denote N the number of data, n_d the dimension of the initial (decoded) space and n_e the dimension of the reduced (encoded) space.

2.3.2 Variational Autoencoders

Treated the concept behind a standard auto-encoder lets us introduce variational autoencoders. The idea behind variational autoencoders is to keep regularity along with the training. To achieve this purpose, we map the latent representation of the input onto a probabilistic distribution instead of a fixed vector.

The regularity of the latent space for autoencoders depends on the distribution of the data in the initial space, the latent space's dimension, and the encoder's architecture. So it is not granted that our model will be able to organize the latent space in a good way compatible with the training process due to the irregularity of the data.

The only difference in a variational autoencoder from the base model is that two separate vectors replace the standard bottleneck vector. One represents the mean of the distribution, and the other represents the standard deviation (see figure below

[**Fig. 2.12**]).

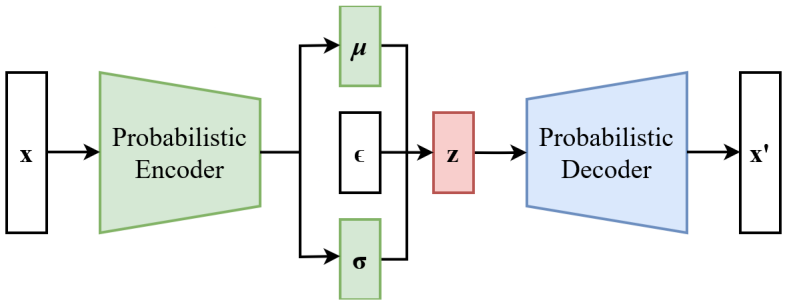


Figure 2.12. The scheme of a variational autoencoder. [40]

The loss function that is minimized when training a VAE is composed of a *"reconstruction term"* that compute the difference between the generated target and the original input, and a *"regularisation term"*, that tends to regularise the distribution over the latent space. That regularisation term is expressed as the Kulback-Leibler divergence[1] between the returned distribution and a standard Gaussian.

2.3.3 Mathematical Formulation

From a formal perspective, given an input x from a dataset X characterized by an unknown probability distribution $P(x)$, the objective is to model or approximate the data's true distribution P using a parametrized distribution p_θ having parameters θ . Let z be a random vector jointly-distributed with x. Conceptually z will represent a latent encoding of x.

$$p_\theta(x) = \int_z p_\theta(x, z)dz$$

where $p_\theta(x, z)$ represents the joint distribution under p_θ of the observable data x and its latent representation or encoding z. According to the chain rule, the equation can be rewritten as

$$p_\theta(x) = \int_z p_\theta(x|z)p_\theta(z)dz$$

[1]In mathematical statistics, the Kullback–Leibler divergence, D_{KL} (also called relative entropy), is a measure of how one probability distribution is different from a second, reference probability distribution. Applications include characterizing the relative (Shannon) entropy in information systems, randomness in continuous time series, and information gain when comparing statistical inference models. In contrast to variation of information, it is a distribution-wise asymmetric measure and thus does not qualify as a statistical metric of spread – it also does not satisfy the triangle inequality. In the simple case, the relative entropy of 0 indicates that the two distributions have identical quantities of information. In simplified terms, it is a measure of surprise, with diverse applications such as applied statistics, fluid mechanics, neuroscience, and bioinformatics.

In the standard formulation of variational autoencoders, x is usually taken to be a finite-dimensional vector of real numbers, and $p_\theta(x|z)p_\theta(z)$ to be a Gaussian distribution. Then $p_\theta(x)$ is a mixture of Gaussian distributions. It is now possible to define the set of the relationships between the input data and its latent representation as

- Prior $p_\theta(z)$
- Likelihood $p_\theta(x|z)$
- Posterior $p_\theta(x|z)$

Unfortunately, the computation of $p_\theta(x)$ is expensive and in most cases intractable. To speed up the calculus to make it feasible, it is necessary to introduce a further function to approximate the posterior distribution as

$$q_\Phi(z|x) \approx p_\theta(z|x)dz$$

with Φ defined as the set of real values that parametrize q. In this way, the overall problem can be easily translated into the autoencoder domain, in which the conditional likelihood distribution $p_\theta(x|z)$ is carried by the probabilistic decoder, while the approximated posterior distribution $q_\Phi(z|x)$ is computed by the probabilistic encoder.

As in every deep learning problem, it is necessary to define a differentiable loss function in order to update the network weights through backpropagation.

For variational autoencoders the idea is to jointly minimize the generative model parameters θ to reduce the reconstruction error between the input and the output, and Φ to have $q_\Phi(z|x)$ as close as possible to $p_\theta(z|x)$.

As distance loss between the two distributions the reverse Kullback–Leibler divergence $D_{KL}(q_\Phi(z|x)||p_\theta(z|x))$ is a good choice to squeeze $q_\Phi(z|x)$ under $p_\theta(z|x)$ [22, 19].

The distance loss just introduced is expanded as

$$
\begin{aligned}
D_{KL}(q_\Phi(z|x)||p_\theta(z|x)) &= \int q_\Phi(z|x) \log \frac{q_\Phi(z|x)}{p_\theta(z|x)} dz \\
&= \int q_\Phi(z|x) \log \frac{q_\Phi(z|x) p_\theta(x)}{p_\theta(z,x)} dz \\
&= \int q_\Phi(z|x) \left(\log p_\theta(x) + \log \frac{q_\Phi(z|x)}{p_\theta(z,x)} \right) dz \\
&= \log p_\theta(x) + \int q_\Phi(z|x) + \log \frac{q_\Phi(z|x)}{p_\theta(z,x)} dz \\
&= \log p_\theta(x) + \int q_\Phi(z|x) + \log \frac{q_\Phi(z|x)}{p_\theta(z|x) p_\theta(z)} dz \\
&= \log p_\theta(x) + \mathrm{E}_{z \sim q_\Phi} \left(\log \frac{q_\Phi(z|x)}{p_\theta(z|x)} - \log p_\theta(z) \right) \\
&= \log p_\theta(x) + D_{KL}(q_\Phi(z|x)||p_\theta(z)) - \mathrm{E}_{z \sim q_\Phi}(\log p_\theta(x|z))
\end{aligned}
$$

At this point, it is possible to rewrite the equation as

$$
\log p_\theta(x) - D_{KL}(q_\Phi(z|x)||p_\theta(z|x)) = \mathrm{E}_{z \sim q_\Phi}(\log p_\theta(x|z)) - D_{KL}(q_\Phi(z|x)||p_\theta(z))
$$

We can define the loss function for our network as

$$
L_{\theta,\Phi} = \mathrm{E}_{z \sim q_\Phi}(\log p_\theta(x|z)) - D_{KL}(q_\Phi(z|x)||p_\theta(z))
$$

Before applying the ELBO loss function to an optimization problem to backpropagate the gradient, it is necessary to make it differentiable by applying the so-called reparameterization trick to remove the stochastic sampling from the formation, and thus making it differentiable.

2.3.4 The Reparameterization Trick

Stochastic sampling is the non-differentiable operation through which it is possible to sample from the latent space and feed the probabilistic decoder.

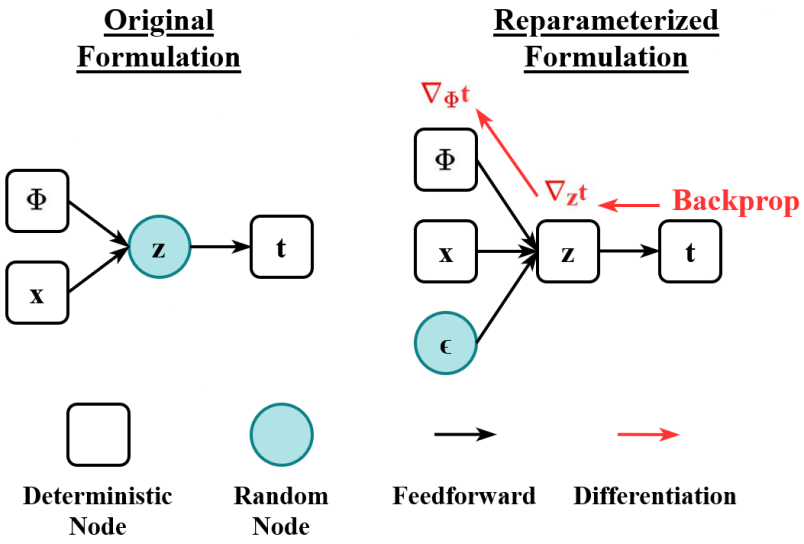


Figure 2.13. The scheme of the reparameterization trick. The randomness variable ϵ is injected into the latent space z as external input. In this way, it is possible to backpropagate the gradient without involving stochastic variable during the update. [40]

The main assumption about the latent space is that it can be considered to be a set of multivariate Gaussian distributions, and thus can be described as

$$z \sim q_\Phi(z|x) = \mathrm{N}(\mu, \sigma^2)$$

Given $\epsilon \sim \mathrm{N}(0, 1)$ and $\odot$ defined as the element-wise product, the reparameterization trick modifies the above equation as

$$z = \mu + \sigma \odot \epsilon$$

Thanks to this transformation (which can be extended to non-Gaussian distributions), the VAE becomes trainable and the probabilistic encoder has to learn how to map a compressed representation of the input into the two latent vectors μ and σ, while the stochasticity remains excluded from the updating process and is injected in the latent space as an external input through the random vector σ.

2.4 Vector Quantization (VQ-VAE)

In Variational Auto Encoders, already covered in the previous section, the idea was to sample images from one distribution and map them to some latent space. Hence taking the probability distribution learned over the latent representation, we can build, starting from the pre-trained network, an inference model, such as a classifier, an image2image synthesis model, a generative model, or a super-resolution model able to increase the details of the image like the shown in the [**Figure 2.14**].

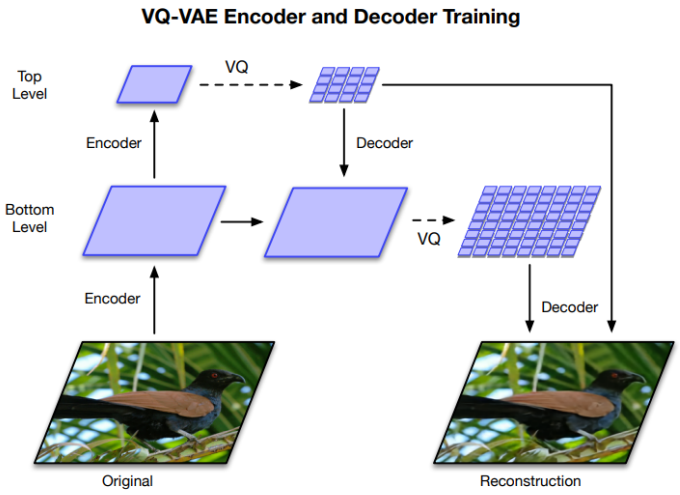


Figure 2.14. Overview of the architecture of our hierarchical VQ-VAE. The encoders and decoders consist of deep neural networks. The input to the model is a 256×256 image that is compressed to quantized latent maps of size 64×64 and 32×32 for the bottom and top levels, respectively. The decoder reconstructs the image from the two latent maps. Source *"Generating Diverse High-Fidelity Images with VQ-VAE-2"* [34]

Most of the data in our world can be represented as an analog signal. This representation can be more or less complex and complete depending on how many features and details we want to take into account. In most cases, to accomplish our task, we do not need to consider and study the data in its entirety. The most simple example is the following: given the task of building a classifier that can distinguish between photos of cats and dogs, we do not care if the model considers the background since we only focus on the subject. For this reason, the background is trivial data and not valid for the scope of the classification. We have already seen how the variational auto-encoder model helps us filter this data throw a reduced latent space of the input.

We can question if is essential to our model to maintains and process data in analog or similar analog signals. A latent natural representation for our data could be that of the natural number field since we are often not interested in focusing our attention on every shades of the object of study. We are interested only in the characteristics useful for its representation that best categorize it for our purposes.

For example, discrete phonemes and terminology can satisfactorily represent human

speech. Additionally, images contain discrete objects with some discrete set of qualifiers. We could imagine having one discrete variable for the type of object, one for its color, one for its size, one for its orientation, one for its shape, one for its texture, one for the background color, one for the background texture, and so on.

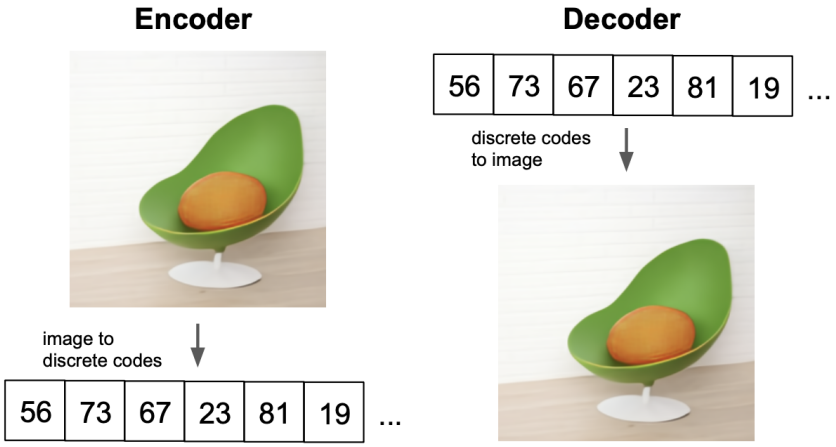


Figure 2.15. image from *"Taming Transformers for High-Resolution Image Synthesis"* [12]

Or to be more precise we can cite the authors. *"Learning representations with continuous features have been the focus of many previous work however we concentrate on discrete representations which are potentially a more natural fit for many of the modalities we are interested in. Language is inherently discrete, similarly speech is typically represented as a sequence of symbols. Images can often be described concisely by language. Furthermore, discrete representations are a natural fit for complex reasoning, planning and predictive learning (e.g., if it rains, I will use an umbrella). While using discrete latent variables in deep learning has proven challenging, powerful autoregressive models have been developed for modelling distributions over discrete variables ."* [38]

2.4.1 Quantizing Autoencoders

The most common technique to quantize a variational auto-encoder is to insert a layer that operates between the encoder and the decoder. To discretize the latent space generated by the encoder and fed to the decoder exist, different techniques, such as clustering algorithm, like *K-Means, K-NN, Random-Forest*, or employ transformers layers.

In our case of study we employ the codebook studied in the article *"Neural Discrete Representation Learning"*, Oord et al. [38] . The model designed by the authors and named VQ-VAE extends the standard variational auto-encoder model adding a discrete codebook component to the network (We can observe the described model in [**Figure 2.16**]).

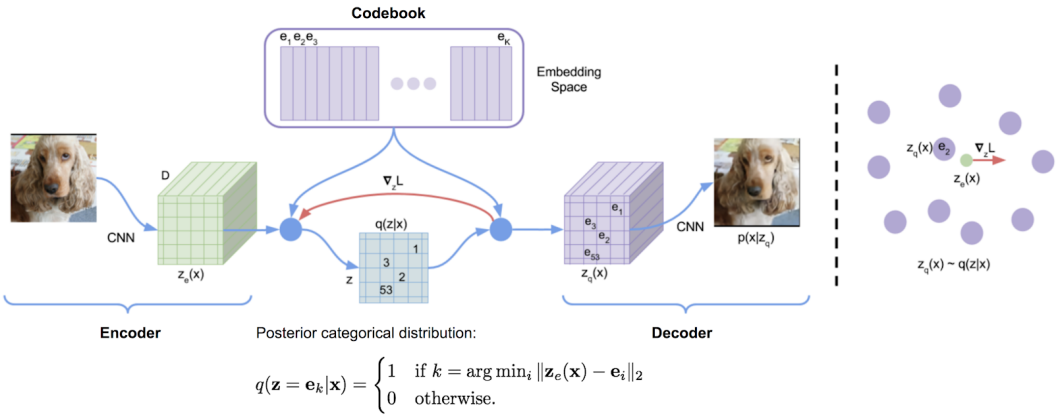
This codebook is a list of vectors associated with a corresponding index used to quantize the network's bottleneck. It works by comparing the encoded input

against all vectors inside the codebook, and the relative codebook vector closest in Euclidean distance is fed to the decoder.

The codebook is defined like a *latent embedding space* $e \in R^{K \times D}$ *where* K *is the size of the discrete latent space (i.e., a* K*-way categorical), and* D *is the dimensionality of each latent embedding vector* e_i*. Note that there are* K *embedding vectors* $e_i \in R_D$*,* $i \in 1, 2, \ldots, K$*. The model takes an input* x*, that is passed through an encoder producing output* $z_e(x)$*.* [38]

Our quantized vectors are computed with the following equation $z_q(x) = \underset{i}{argmin} ||z_e(x) - e_i||_2$, where $z_e(x)$ is the encoder vector for some raw input x, e_i is the i-th codebook vector, and $z_q(x)$ is the resulting quantized vector that is passed as input to the decoder.

The fundamental difference between a VAE and a VQ-VAE resides in the first one that learns a continuous latent representation, while VQ-VAE learns discrete ones.



$$q(\mathbf{z} = \mathbf{e}_k | \mathbf{x}) = \begin{cases} 1 & \text{if } k = \arg\min_i \|\mathbf{z}_e(\mathbf{x}) - \mathbf{e}_i\|_2 \\ 0 & \text{otherwise.} \end{cases}$$

Figure 2.16. Left: A figure describing the VQ-VAE. Right: Visualisation of the embedding space. The output of the encoder $z(x)$ is mapped to the nearest point e_2. The gradient $\Delta_z L$ (in red) will push the encoder to change its output, which could alter the configuration in the next forward pass. Source : *"Neural Discrete Representation Learning"* [38]

We can notice how immediately catches the eye the *argmin* operation performed to build the codebook, since it is non-differentiable with respect to the encoder(see figure above [**Fig. 2.16**]). But this obstacle can be overcame if we pass the decoder gradient directly through this operation to the encoder (i.e. set its gradient to 1 *w.r.t.* the encoder and the quantized codebook vector; and to 0 *w.r.t.* all other codebook vectors)

$$q(z = k|x) = \begin{cases} 1 & \text{for} \quad k = \underset{j}{argmin} ||z_e(x) - e_k||_2 \\ 0 & \text{otherwise} \end{cases}$$

We view this model as a VAE in which we can bound $\log p(x)$ *with the ELBO. Our proposal distribution* $q(z = k|x)$ *is deterministic, and by defining a simple uniform prior over* z *we obtain a* KL *divergence constant and equal to* $\log K$*. The representation* $z_e(x)$ *is passed through the discretisation bottleneck followed by mapping onto the nearest element of embedding* e*.* [38]

$$z_q = e_k, \quad \text{where} \quad k = \underset{j}{argmin} ||z_e(x) - e_k||_2$$

We then task the decoder to reconstruct the input from this quantized vector as in the standard autoencoder formulation.

Recall that VAEs, to reconstruct an image from the relative latent distribution, proceeds as follows: the encoder is tasked with approximating the posterior distribution of the latent $p(z|x)$, while the decoder approximates the reconstruction from the latent space $p(x|z)$.

VQ-VAE assumes a uniform prior to all the latent codes during training, so all latents are equally likely. Additionally the posterior is deterministic in that $p(z|x) = 1$ if $z = z_q(x)$ and 0 otherwise. Lastly, the decoder distribution $p(x|z)$ is unchanged and learned as usual. Using these definitions, the *KL divergence* term that usually appears in the ELBO is constant w.r.t. the encoder parameters and can thus be ignored for training, leaving just the reconstruction term.

2.4.2 Learning the Codebook

We can study how the codebook is learned by observing the overall loss function since codebook vectors are tuned during the backpropagation operation while we go to update the learning weights of our model. Three components employed to train different parts of our VQ-VAE characterized the loss function. The first term is the *reconstruction loss* which optimizes the decoder and the encoder. As we mentioned before, since the notebook uses a discrete representation, the gradient cannot pass through it, and for this reason, we resort to the skip gradient escamotage.

"Therefore, in order to learn the embedding space, we use one of the simplest dictionary learning algorithms, Vector Quantisation (VQ). The VQ objective uses the $l2$ *error to move the embedding vectors* e_i *towards the encoder outputs* $z_e(x)$*"* [38] as shown in the second term of the loss equation below.

$$L = \log(p(x|q(x))) + ||\text{sg}[z_e(x)] - e||_2^2 + \beta||z_e(x) - \text{sg}[e]||_2^2$$

The stop gradient operator $sg[\cdot]$ [2] on the third term is meant only to update the codebook. Its scope is to get the encoder output to commit to its closest codebook vector as much as possible. This term is called the *codebook commitment loss*, and its importance to the overall loss is scaled by the tunable hyperparameter β.

With this in mind, we can fully train our VQ-VAE, making it capable of reconstructing a diverse set of inputs that are almost imperceptibly different from the original. As we can see, the figure below Figure [**Fig. 2.17**] that shows a sample of images with their ground truth values on the left and the reconstructed ones on the right.

.

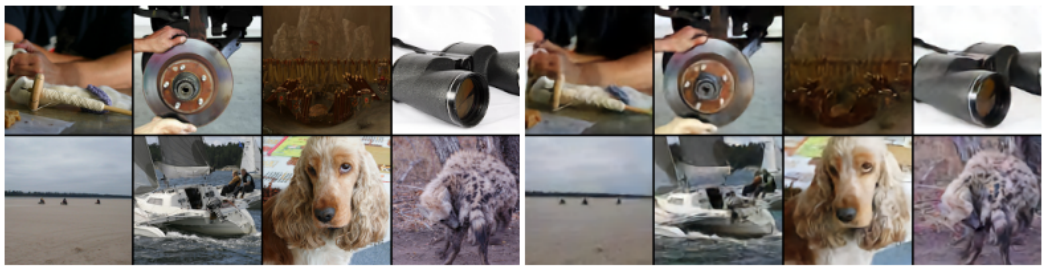


Figure 2.17. left: ImageNet 128x128x3 images, right: reconstructions from a VQ-VAE with a $32 \times 32 \times 1$ latent space, with $K = 512$. Source : *"Neural Discrete Representation Learning"* [38]

[2] $sg[\cdot]$: it stands for stop-gradient or skip-gradient. This operation is performed to avoid errors coming from the discrete non derivable nature of the vector-quantization layer.

2.5 Linear Attention Mechanism (LAM)

The modern attention mechanism, which is nowadays in use as the key component for several works in our sector, was introduced from the article *"Attention is all you need"*[39], Vaswani et al. . The article is very popular and known since it gave birth to the transformers model. The idea behind the attention mechanism was to permit the utilization of the most relevant parts of the input sequence flexibly, by a weighted combination of all of the encoded input vectors, with the most relevant vectors being attributed the highest weights. It can meticulously refine the feature maps and boost the network's performance.

Over the years, it has become an essential technology applied in computer vision and natural language processing. However, the memory and computational costs of the dot-product attention mechanism, core operation of the process, widely used in vision and language tasks, increase quadratically with the spatio-temporal size of the input. Such growth hinders the usage of the attention mechanism in application scenarios with large inputs. New strategies are emerging to reduce computational costs; a very promising ones is the linear attention mechanism (LAM), illustrated in the article *"Linear Attention Mechanism: An Efficient Attention for Semantic Segmentation"* [26] Li et al., that presents an alternative attention-based calculation to reduce the computational cost from $O(N^2)$ to $O(N)$.

2.5.1 Definition of Dot-Product Attention

Given the brief introduction, we can now see how it works starting from defining the attention mechanism designed by Vaswani et al. [39]. Providing N and C as the length of input sequences and the number of input channels, where $N = H \times W$, with H and W the height and width of the input, with the input feature $X = [x_1, ..., x_N] \in R^{N \times C}$, dot-product attention utilizes three projected matrices $W_q \in R^{D_x \times D_k}$, $W_k \in R^{D_x \times D_k}$, and $W_v \in R^{D_x \times D_v}$ to generate the corresponding query matrix Q, the key matrix K, and the value matrix V

$$Q = XW_q \in R^{D_x \times D_k}$$
$$K = XW_v \in R^{D_x \times D_v}$$
$$W = XW_v \in R^{D_x \times D_v}$$

Note the dimensions of the query matrix and key matrix have to be identical and all the vectors in this letter are column vectors by default. A normalization function ρ that evaluates the similarity between the i_{th} query feature $q_i^T \in R^{D_k}$ and the j_{th} key feature $k_j \in R^{D_k}$ by $\rho(q_i^T k_j) \in R^1$. Generally, as the query feature and key feature are generated by diverse layers, the similarities between $\rho(q_i^T k_j)$ and $\rho(q_j^T k_i)$ are not symmetric. By calculating the similarities between all pairs of positions and taking the similarities as weights, the dot-product attention module computes the value at position i by aggregating the value features from all positions based on the weighted summation

$$D(Q, K, V) = \rho(QK^T)V$$

The softmax is taken as the common normalization function

$$\rho(QK^T)V = \text{softmax}_{row} \quad (QK^T)$$

where $softmax_{row}$ indicates applying the softmax function along each row of matrix QK^T . $\rho(QK^T)$ models the similarities between each pair of pixels of the input, thoroughly extracting the global contextual information contained in the features.

Hence according to Vaswani et al. [39] the attention is computed as follows:

$$D(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)$$

Where d_k is a scale factor. As $Q \in R^{N \times D_k}$ and $K^T \in R^{D_k \times N}$, the product between Q and K^T belongs to $R^{N \times N}$, which leads to $O(N^2)$ memory and computational complexity.

2.5.2 Generalization of Dot-Product Attention

Under the condition of softmax normalization function, the i_{th} row of result matrix generated by the dot-product attention module can be written as

$$D(Q, K, V)_i = \sum_{j=1}^{N} \frac{e^{q_i^T k_j} v_j}{e^{q_i^T k_j}}$$

The equation above can be further generalized as follows:

$$D(Q, K, V)_i = \sum_{j=1}^{N} \frac{sim(q_i^T k_j) v_j}{sim(q_i^T k_j)}$$
$$sim(q_i^T k_j) > 0$$

We can notice that if we define $sim(q_i^T k_j) = e^{q_i^T k_j} v_j$ our generalized equation lead to the softmax relative ones. Reached this point we can expand the $sim(q_i^T k_j)$ term as $\phi(q_i)^T \varphi(k_i)$, which measures the similarity between q_i and k_j. Specifically, if $\phi(\cdot) = \varphi(\cdot) = e^{(\cdot)}$, is equivalent to the attention mechanism design by Vaswani et al. [39]. Under this we can rewrite our generalised equation in

$$D(Q, K, V)_i = \sum_{j=1}^{N} \frac{\phi(q_i)^T \varphi(k_i) v_j}{\phi(q_i)^T \varphi(k_i)}$$
$$sim(q_i^T k_j) > \phi(q_i)^T \varphi(k_i)$$

And can be further simplified as

$$D(Q,K,V)_i = \frac{\phi(q_i)^T \sum_{j=1}^{N} \varphi(k_i) v_j}{\phi(q_i)^T \sum_{j=1}^{N} \varphi(k_i)}$$

As $K \in R^{D_k \times N}$ and $V^T \in R^{N \times D_v}$, the product between K and V^T belongs to $R^{D_k \times D_v}$, which reduces the complexity of the dot-product attention mechanism considerably.

2.5.3 Linear Attention Mechanism

As discussed above, talking of the dot-product attention mechanism, we have exhibited that it has a computational cost of $O(N^2)$. Faced with this required computational effort, which turns into a considerable computational power required, especially in larger models, which leads to extreme processing times, the Linear Attention Mechanism (LAM) proposes to reduce this effort. The Linear Attention Mechanism arises through an approximation of first-order Taylor's series expansions to modify the standard attention mechanism to get a new one with a computational cost of $O(N)$.

$$e^{q_i^T k_j} \approx 1 + q_i^T k_j$$

The above approximation, however, cannot guarantee non-negativity. To ensure $q_i^T k_j \geq -1$, we can normalize q_i and k_j by the $l2$-norm.

$$q_i^T k_j = 1 + \left(\frac{q_i}{||q_i||_2}\right)^T \left(\frac{k_j}{||k_j||_2}\right)$$

Then we can rewrite the dot product attention as follows:

$$D(Q,K,V)_i = \frac{\sum_{j=1}^{N} \left(1 + \left(\frac{q_i}{||q_i||_2}\right)^T \left(\frac{k_j}{||k_j||_2}\right)\right) v_j}{\sum_{j=1}^{N} \left(1 + \left(\frac{q_i}{||q_i||_2}\right)^T \left(\frac{k_j}{||k_j||_2}\right)\right)}$$

Hence the equation can be simplified as

$$D(Q,K,V)_i = \frac{\sum_{j=1}^{N} v_j + \left(\frac{q_i}{||q_i||_2}\right)^T \sum_{j=1}^{N} \left(\frac{k_j}{||k_j||_2}\right) v_j^T}{N + \left(\frac{q_i}{||q_i||_2}\right)^T \sum_{j=1}^{N} \left(\frac{k_j}{||k_j||_2}\right)}$$

The equation can be transformed into a vectorized form.

$$D(Q,K,V) = \frac{\sum_j V_{ij} + \left(\frac{Q}{||Q||_2}\right)\left(\left(\frac{K}{||K||_2}\right)^T V\right)}{N + \left(\frac{Q}{||Q||_2}\right)\sum_j \left(\frac{K}{||K||_2}\right)^T_{ij}}$$

As $\sum_{j=1}^{N}((k_j/(||k_j||_2))v_j^T$ and $\sum_{j=1}^{N}((k_j)/(||k_j||_2))$ is common to every query, the complexity of this linear attention mechanism is $O(N)$. Nevertheless, for the channel dimension dot-product-based attention is computed, considering that the channels of the input C are much less than the pixels.

2.6 MAResUNet

As the foundation for the design of the model presented in this thesis, we followed the same strategy of the article *"MARE: Self-Supervised Multi-Attention REsu-Net for Semantic Segmentation in Remote Sensing"* Masocci et al. [33]. They employ the network configuration conceived by Li et al. in their work *"Multistage attention ResU-Net for semantic segmentation of fine-resolution remote sensing images"* [27] (see **[Fig. 2.18]** left). The model uses the ResUNet framework, where skip-connections link the blocks between encoder and decoder and integrate custom-attention mechanism layers called LAM (Linear Attention Mechanism Module) [25].

These modules (see **[Fig. 2.18]** right), which constitute an alternative to traditional Transformers, have been implemented to help the network capture the relational information between the blocks. The LAM mechanism has been introduced in place of the classic attention mechanism to reduce computational complexity due to the high cost of the dot-product attention calculation and, consequently, the computation times.

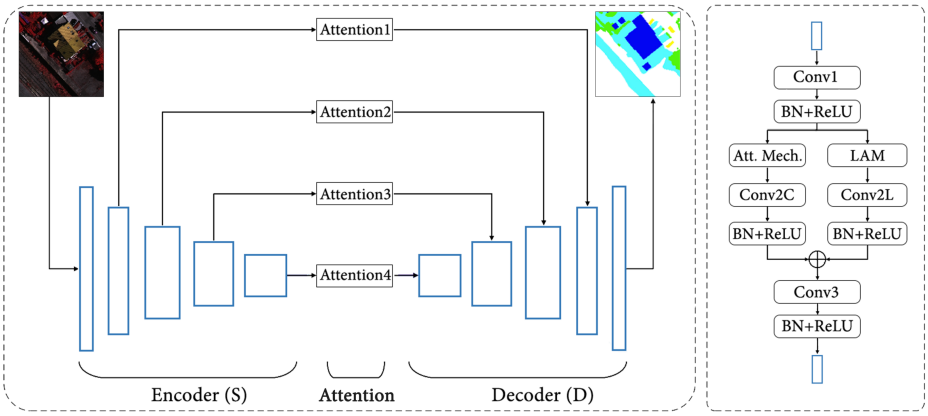


Figure 2.18. MAResU-Net architecture (left) and MAResU-Net attention block (right). [33]

Based on the LAM [25] Proposal, the authors redesign the skip connections of the original U-Net and design a ResU-Net with multistage attention layers (MAResU-Net). The acronym MAResUNet stands for Multistage Attention ResU-Net. It is the merge of ResNet, short for Residual Network, a specific type of convolutional neural network that was introduced in 2015 by He et al. in their article *"Deep Residual Learning for Image Recognition"* [18], And UNet which is a framework proposed by the document *"U-Net: Convolutional Networks for Biomedical Image Segmentation"* by Ronneberger et al. (see **[Fig. 2.19]**) [35].

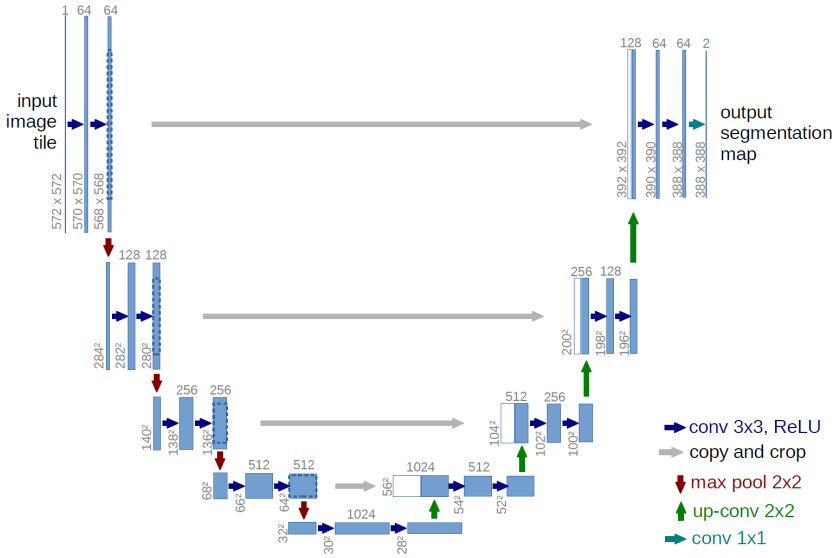


Figure 2.19. U-net architecture (example for 32×32 pixels in the lowest resolution). Each blue box corresponds to a multi-channel feature map. The number of channels is denoted on top of the box. The x-y-size is provided at the lower left edge of the box. White boxes represent copied feature maps. The arrows denote the different operations. [35]

In this network configuration, the encoder and decoder share the same architecture, where the only difference consists of the decoder having the same mirrored architecture as the encoder. The decoder comprises four macro-blocks composed of various atrous convolutional layers, which halve the input dimensions as they go downwards. Each convolutional layer employs a 3×3 sized kernel. Pooling layers are placed between the various macro-blocks, and a pooling layer is placed between one macro-block and the other to perform the down-scale in the case of the encoder while an up-scaling in the case of the decoder. Since we are talking about a ResNet between the macro-blocks before passing through the activation function, a ReLu (Rectifier Linear Units), the resultant of the pooling of the block is added to the resultant of the LAM layers.

Chapter 3

Pre-Training Techniques

In the fields of deep neural networks during the past years, much more attention has been given to the structure of the model and the various types of layers implemented than to other factors such as the balance of parameters and training methods.

Recently, several works have put more and more focus on the training procedures. The common practice is to pre-train a model to learn the prior distribution of a given dataset and then use the same model parameters to start training a new one.

In the case study of supervised-learning we force the variational auto-encoder to learn to reconstruct a distorted or augmented input. We can figure this pre-training like the definition of a denoiser model.

Once we have obtained a model capable of recreating the starting input, we extract the encoder, deeming it capable of an effective latent representation of the source dataset.

Starting from this model, it is now possible to extract the encoder parameters, which by now acquired a good ability to represent the latent space of the initial dataset, with the aim of re-train the network to extract types of data always associated with the input. As in the case of interest of our thesis, these new targets can be the segmentation masks of the regions of satellite images.

It is now better to ask whether it is more convenient to train the neural network directly with the target images. The answer is that very often, as previously stated in the previous sections, there is not always a wide range of data available to draw upon. Direct training of the model that takes the photos as input and synthesizes them into masks can therefore fall very easily under the effect of model overfitting phenomena.

Over time, other pre-training methodologies have been developed to obtain increasingly efficient models. Among these techniques, the *online self-distillation learning* approach is emerging with increasing popularity, derived from breaking record after record in computer vision benchmarks.

This procedure is the fusion of three methodologies that all share the use of two neural networks during training. Although initially designed for different purposes, it was found that the strategies applied very well together over time. These training procedures are called: *knowledge-distillation*, *online distillation*, and *self-distillation*.

However, before discussing these types of training, we would prefer to take a step back and see where the term *distillation* comes from and how it relates to neural networks

This pre-training methodology has its roots in the work of Hinton et al. Presented in the article *"Distilling the knowledge in a neural network"* [20]. In their work Hinton et al. set the goal of reducing the size of the neural networks without losing the accuracy of the model. The larger the model, the greater the precision. Nevertheless, as the size increases, the computation load increases in the inference and training phases. Consequently, by defining a smaller size model capable of using the distribution of a larger network in the same way, it is possible to obtain a substantial saving of resources in the inference phase.

The mechanism involved taking two neural networks: a large one with a huge number of parameters is defined as the teacher network, and a smaller one with a small number of parameters is defined as the student network. As a first step, the teacher network, one of the larger dimensions, is trained in supervised learning. Later that same network is used to train the smaller one, our student network, using the distribution that the former learned during its training. This technique took the name of *Knowledge-Distillation learning*.

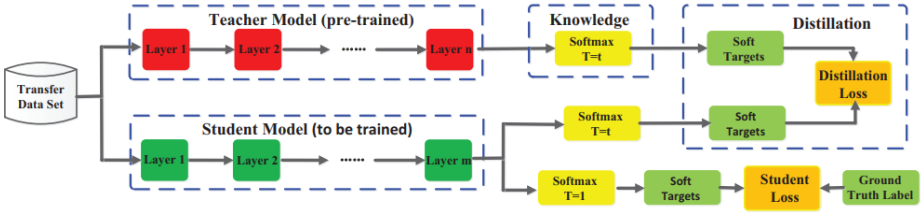


Figure 3.1. The specific architecture of the benchmark knowledge distillation (Hinton et al., 2015 *"Distilling the knowledge in a neural network"* [20]). Source *"Knowledge distillation: A survey"* Gou et al. [15].

This result can be obtained by following the training method shown in figure [**Fig 3.1**], where two components are combined in the student network loss function: the *"Distillation loss"* and the *"Student Loss"*. The *"Distillation loss"* is obtained by comparing the predictions of the network being trained with the distribution resulting from the teacher's network (The predictions of the teacher's network are defined as *soft-targets*). The *"Student Loss"* derives from the comparison calculation between student output and the ground-truth targets (defined as *hard-targets*).

Keeping in mind the functioning of this training mechanism, let us go back to

talking about the previously mentioned technique, *"online self-distillation"*, and let us study how it differs from the one just discussed.

As we did before, let us break down the method to be studied into *online-distillation* and *self-distillation*. Remember that to follow the *Knowledge-Distillation* framework is necessary pre-train a large neural network (teacher) to make use of it during the training of the smaller one (student). While in *online-distillation* both networks, the teacher and the student are updated simultaneously. This paradigm was initially studied in the works of Zhang et al. In the article *"Deep mutual learning"* [41] and by Anil et al. [1] in the article *"Large scale distributed neural network training through online distillation"*. The study aimed to overcome the limitations of offline distillation, proposing *online-distillation* to improve the student network's performance further, especially when a high-performance teacher network model is not available.

Having discussed the methods of *knowledge-distillation* and *online-distillation*, we can move on to the *self-distillation* approach. The method called *self-distillantion* was born as an extension of *online-distillation*.

The strength of self-distillation learning lies in combining labeled and unlabeled samples to expand the pool of data available for model training. Consequently, it makes it possible to improve the model's training by reducing the load and data required for training by not resorting to huge data sets already labeled and cataloged.

This technique arises from the necessity of processing huge amounts of data not yet labeled, and that labeling manually would be costly in terms of time and human resources. Taking as an example the data treated in our thesis, those relating to satellite images, imagine how long and how many people are needed to extract the segmentation masks for an entire city.

So this method allows us to pre-train a network with a large amount of unlabeled data, to obtain an encoder capable of capturing a significant latent representation. Once this encoder has been obtained, we can use it to train a new network even with a small set of labeled data, avoiding overfitting phenomena from occurring.

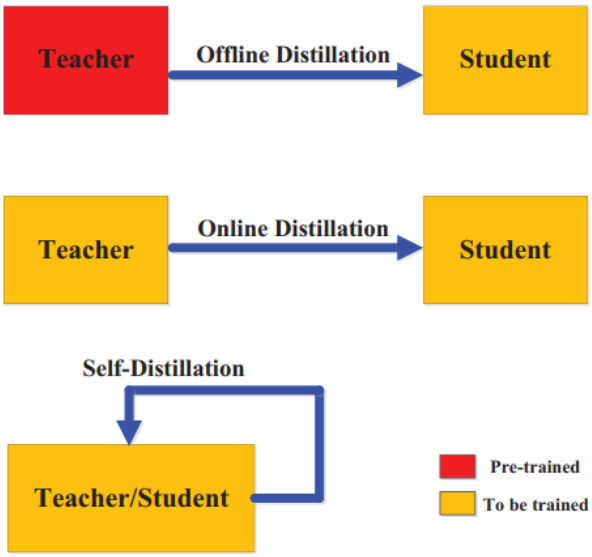


Figure 3.2. Different distillations. The red color for *"pre-trained"* means networks are learned before distillation and the yellow color for *"to be trained"* means networks are learned during distillation. Source *"Knowledge distillation: A survey"* Gou et al. [15].

Now that we have clarified the objectives of this methodology we can proceed to explain how it works.

Self-distillation is a learning paradigm where we train a student network g_{θ_s} to match the output of a given teacher network g_{θ_t} , parameterized by θ_s and θ_t respectively. Given an input image x, both networks output probability distributions over K dimensions denoted by P_s and P_t. The probability P is obtained by normalizing the output of the network g with an softmax function.

Given a fixed teacher network g_{θ_t}, we learn to match these distributions by minimizing the loss w.r.t. the parameters of the student network θ_s.

Reached this stage, we have to update the teacher network. To fulfill this, we cannot merely back-propagate the loss via the teacher, or we will train two different networks, in the same way, ending with a clone of the student. To avoid this scenario, we freeze the parameters of the teacher, and instead of back-propagate the error through it, we update the teacher through a momentum-encoder using the EMA (exponential moving average) based on the student parameters (see figure below [**Fig. 3.3**]).

$$\theta_T = \alpha\theta_T + (1-\alpha)\theta_S$$

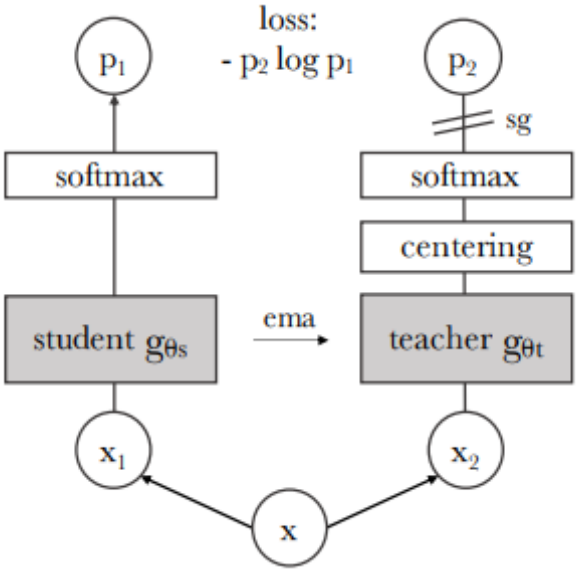


Figure 3.3. Both networks have the same architecture but different parameters. The output of the teacher network is centered on a mean computed over the batch. Each network outputs a K dimensional feature normalized with a temperature softmax over the feature dimension. Their similarity is then measured with a cross-entropy loss. We apply a stop-gradient (sg) operator on the teacher to propagate gradients only through the student. The teacher parameters are updated with an exponential moving average (ema) of the student parameters. [7]

Let us begin by explaining what an EMA (Exponential Moving Average) is and in which fields it is used. An EMA is often used in the technical analysis of financial data, like stock prices, returns, or trading volumes. It is also used in economics to examine the gross domestic product, employment, or other macroeconomic time series. Mathematically, a moving average is a type of convolution, and so it can be viewed as an example of a low-pass filter used in signal processing.

We can define exponential moving average (EMA) as type a of moving average (MA) that places a greater weight and significance on the most recent data points. The exponential moving average is also referred to as the exponentially weighted moving average. Formulated as follows:

$$EMA_{today} = Value_{today} * \left(\frac{smoothing}{1 + Days}\right) + EMA_{yesterday}\left(1 - \left(\frac{smoothing}{1 + Days}\right)\right)$$

An EMA does serve to alleviate the negative impact of lags to some extent. Because the EMA calculation places more weight on the latest data, it “hugs” the price action a bit more tightly and reacts more quickly.

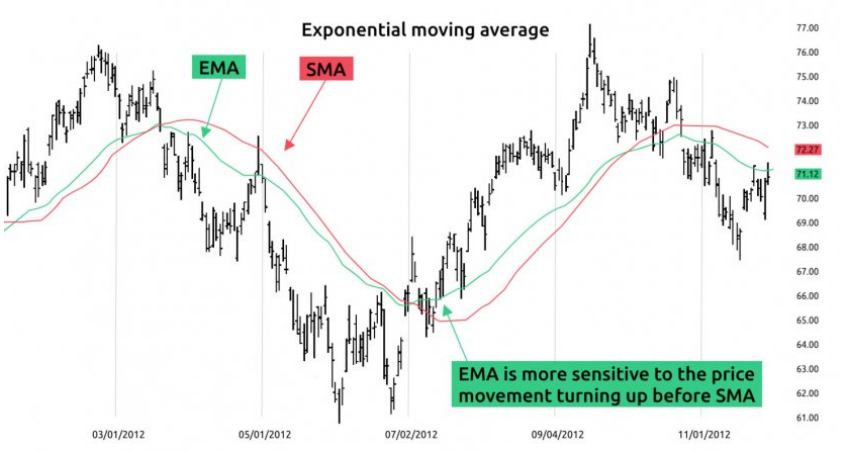


Figure 3.4. Exponential moving average curves example. [21]

It is immediate to catch how the teacher network can support its twin, the student, to predict the new "trends" of the parameter weights, avoiding spiking in the training cycle. This procedure leads to a more regular training process that will reflect the results in improving the accuracy score of our model.

3.1 OBoW

Among the techniques mentioned above, let us now talk about OBoW. OBoW stands for *"Online Bag of Word"* is a self-distillation technique for unsupervised-learning based on a teacher-student scheme to learn the representations of a latent space by training a convolutional network to reconstruct a bag of visual words (BoW) representation of an image.

This methodology, in its reference article, shows how on the reference dataset ImageNet, it is possible to overcome in the benchmark phases other similar self-distillation approaches as they are studied on the contrastive learning method currently MoCo *"Momentum contrast for unsupervised visual representation learning"* ([17]), SimCLR (*"A simple framework for contrastive learning of visual representations"* [9]), SwAV (*"Unsupervised learning of visual characteristics through assignments of contrasting clusters"* [5]), BYOL (*"Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning"* [16])

The pre-training technique called OBoW (Online Bag of Words), [14] was involved in boosting the model performance of the one used as reference in this thesis.

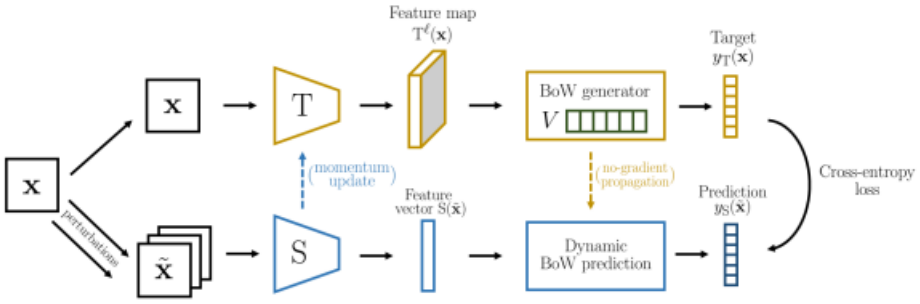


Figure 3.5. Unsupervised learning with Bag-of-Words guidance. Two encoders T and S learn at different tempos by interacting and learning from each other. An image x is passed through the encoder T and its output feature maps $T^l(x)$ are embedded into a BoW representation $y_T(x)$ over a vocabulary V of features from T. The vocabulary V is updated at each step. The encoder S aims to reconstruct $y_T(x)$ from data-augmented instances $\tilde{x}$. A dynamic BoW-prediction head learns to leverage the continuously updated vocabulary V to compute the BoW representation from the features $S(\tilde{x})$. T follows slowly the learning trajectory of S via momentum updates. [14]

It utilizes a online self-supervised learning process to extract a latent representation of the input. Hence detaching and re-using the same encoder process to fine-tune a new VAE (Variational Auto-Encoder) built over the MAResUNet architecture.

The bag-of-words training process works as follows; the network is tasked to reconstruct an input from different augmented samples produced from the original image. The augmented samples produced are categorized in global augmented views, that represents the teacher inputs, and local augmented view, which are fed to the student. The training flow of the algorithm can be observed in the figure above

[Fig. 3.5]. A student convnet $S(\cdot)$ learns image representations, and a teacher convnet $T(\cdot)$ generates BoW targets used for training the student network. The student $S(\cdot)$ is parameterized by θ_S and the teacher $T(\cdot)$ by θ_T. To generate a BoW representation $y_T(x)$ out of an image x, the teacher first extracts the latent space representation of the image using the network like an encoder, hence it quantizes the c-dimensional feature vectors $T^l(x)[u]$ at each location $u \in \{1, \ldots, h_l \times w_l\}$ of the feature map over a vocabulary $V = [v_1, \ldots, v_K]$ of K visual words of dimension c_l.

The vocabulary V of visual words is a K-sized queue of random features. At each step, after computing the assignment codes over the vocabulary V, V is updated by selecting one feature vector per image from the current mini-batch, removing the oldest item in the queue if its size exceeds K. The feature selection consists of a local average pooling with a 3×3 kernel of the feature map $T^l(x)$ followed by a uniform random sampling of one of the resulting feature vectors. Thus, assuming that the local features in a 3×3 neighborhood belong to one common visual concept, local averaging selects a representative visual-word feature from this neighborhood.

This quantization process produces for each location u a K-dimensional code vector $q(x)[u]$ that encodes the assignment of $T(x)[u]$ to its closest (in terms of squared Euclidean distance) visual word(s). Then, the teacher reduces the quantized feature maps $q(x)$ to a K-dimensional BoW, $\tilde{y}_T(x)$, by channel-wise max-pooling, i.e., $\tilde{y}_T(x) = \max_u \quad q(x)[u][k]$ (alternatively, the reduction can be performed with average pooling), where $q(x)[u][k]$ is the assignment value of the code $q(x)[u]$ for the k-th word. Finally, $\tilde{y}_T(x)$ is converted into a probability distribution over the visual words by $L1$-normalization, i.e. $y_T(x)[k] = \frac{\tilde{y}_T(x)[k]}{\sum_{k'} \tilde{y}_T(x)[k']}$.

To learn image representations, the student gets as input a perturbed version of the image x, denoted as $\tilde{x}$, and is trained to reconstruct the BoW representation $y_T(x)$, produced by the teacher, of the original unperturbed image x, taken as target. To that end, the student first extracts a global vector representation $S(\tilde{x}) \in R^c$ (with c channels) from the entire image $\tilde{x}$ and then applies a linear-plus-softmax layer to $S(\tilde{x})$, as follows:

$$y_S(\tilde{x})[k] = \frac{exp(w_k^T S(\tilde{x}))}{\sum_{k'} exp(w_{k'}^T S(\tilde{x}))}$$

where $W = [w_1, \ldots, w_K]$ are the c-dimensional weight vectors (one per word) of the linear layer. The K-dimensional vector $y_S(\tilde{x})$ is the predicted softmax probability of the target $y_T(x)$. Hence, the training loss that is minimized for a single image x is the cross-entropy loss between the softmax distribution $y_S(\tilde{x})$ predicted by the student from the perturbed image $\tilde{x}$, and the BoW distribution $y_T(x)$ of the unperturbed image x given by the teacher.

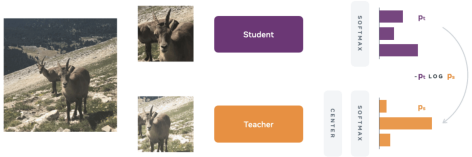
3.2 Benefits of Data Augmentation

Previously we have mentioned on image augmentation to feed different perturbed version of the same image $\tilde{x}$ as input for the student and the teacher network. From a given image, we generate a set V of different views.

This set contains views of two different resolutions $V = [V_g, V_l]$, where $V_g = \{\tilde{x}_{gi} | i = 1, 2\}$ is a global-view set of higher resolution, and $V_l = \{\tilde{x}_{l_i} | i = 1, \ldots, 8\}$ is a local-view set of lower resolution. All views V are passed through the student while only the global views V_g are passed through the teacher.



(a) Reconstructing BoWs from small parts of the original image. [14]



(b) image from DINO augmentation [7]

Figure 3.6. Data augmentation examples

Given a training image (left) [fig. 3.6a] , we extract two types of image crops. The first type (middle)[fig. 3.6a] is obtained by randomly sampling an image region whose area covers no more than 60% of the entire image, resizing it to 160×160 and then giving it as input to the student as part of the reconstruction task. The second type (right) is obtained by randomly selecting an area that covers from 60% to 100% of the entire image, resizing it to a 256×256 , dividing it into 3×3 overlapping patches of size 96×96, and randomly choosing 5 out of these 9 patches (indicated with red rectangles) that are given as 5 separate inputs to the student. The student must then reconstruct the original BoW target independently for each patch. The blue rectangle on the left image indicates the central 224×224 crop from which the teacher produces the BoW target. Note that, except horizontal flipping, no other perturbation is applied to the teacher's inputs. By contrast in other works like DINO [7] and EsViT [36], perturbation of the image helps the networks to understand better the relevant features. In the paper MARE [33] it was shown that the perturbation does not help when it comes to analyzing remote sensing images, even if the leads to a better result when we go to work and study benchmark datasets like ImageNet and COCO (Common Objects in Context).

Chapter 4

Proposed Models

The study presented in this work is a continuation of the starting article MARE [33]. In the study by Masocci et al., we have seen that the use of a pre-train in the form of self-distillation has led to an improvement in the pixel accuracy metric. This approach applied to the I.S.P.R.S. Vaihingen, having a smaller extension than the Potsdam dataset discussed in this thesis, was reproduced for the larger Potsdam dataset. It should be noted that a difference in the extension of the soil translates into the processing of patches in a different number of available samples, a topic set out in detail in the next section relating to data processing.

The first experiments were conducted on two different versions of the model proposed in the paper *"Multistage Attention ResU-Net for Semantic Segmentation of Fine-Resolution Remote Sensing Images"* by Li et al. [28] then reused in MARE [33]. The proposed model was utilized without variation in the first tests using two different architectures as backbone: ResNet18 and ResNet34. In the second version, use was made of the framework proposed in the article *"Neural Discrete Representation Learning"* [38], which combined with our already defined architecture has led to a model based on the VQ-VAE concept using attention mechanisms. The vector quantization technique was introduced due to its presence in the OBoW pre-training method and due to its considerable success deriving from the work related to VQ-GAN *a.k.a. "Taming Transformers for High-Resolution Image Synthesis"* [12] which focuses its interest in images super-resolution and image style transfer. Even these latest models that utilize the vector quantization technique have been tested with ResNet18 and ResNet34 as backbones.

The first testing phase was carried out using a transfer learning with loaded ImageNet weights; subsequently, the tests were carried out with the pre-training techniques from the article "Efficient Self-supervised Vision Transformers for Representation Learning," a.k.a. DINO performed on the training set. All training data and self-distillation methods are covered in the following sections. This self distillation technique later reused in the article "Training data-efficient image transformers & distillation through attention" a.k.a EsViT [36], has shown to obtain better results compared to OBoW in the COCO and ADE20K benchmarks. The two articles cited above concerning self distillation techniques, although concentrating heavily on

the use of transformers models such as DeiT (*"Training data-efficient image transformers & distillation through attention"* [37] Touvron et al.), and SwinT (*"Swin transformer: Hierarchical vision transformer using shifted windows"* Liu et al. [30]) have shown that tests on Resnet models have also led to promising results.

4.1 Dataset

4.1.1 Data description

The society
The I.S.P.R.S. international company provided the dataset on which the work of this thesis is based for photogrammetry and remote sensing. It is a non-governmental organization dedicated to developing international cooperation for the advancement of photogrammetry and remote sensing and their applications.

The Dataset

The dataset used is that relating to the city of Potsdam. It shows a typical historical city with large blocks of buildings, narrow streets, and dense settlement structures.

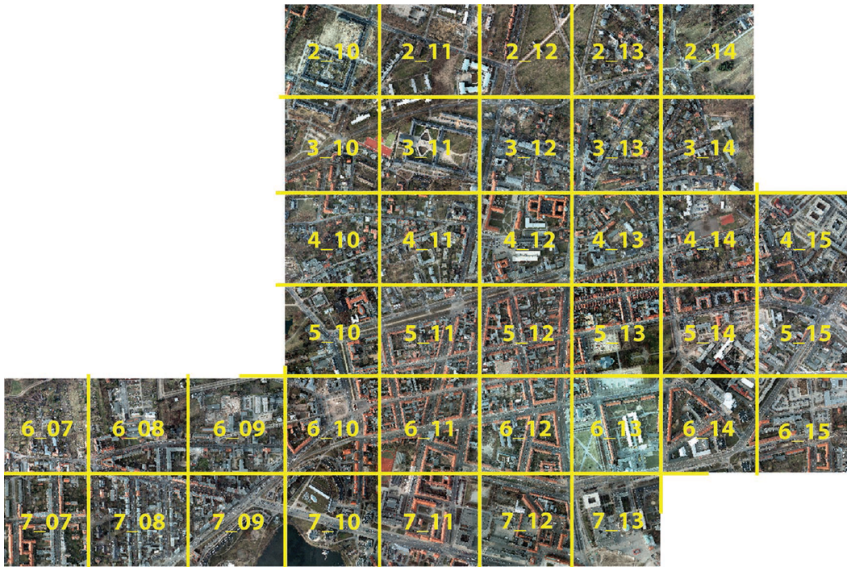


Figure 4.1. Outlines of all patches given in Tab.1 overlaid with the true orthophoto mosaic. Numbers refer to the individual patch numbers, encoded in the filenames as well

The dataset was provided in the form of a tessellated map divided into macroblocks. Each of the 38 macroblocks of the map consists of high-resolution (6000x6000 pixel) treue orthophoto (TOP) images. Each area was associated with a semantic segmentation mask manually classified into six land cover classes.

Six categories/classes have been defined:

1. Impervious surfaces (RGB: $255, 255, 255$)
2. Building (RGB: $0, 0, 255$)
3. Low vegetation (RGB: $0, 255, 255$)
4. Tree (RGB: $0, 255, 0$)

5. Car (RGB: $255, 255, 0$)
6. Clutter/background (RGB: $255, 0, 0$)

The clutter/background class includes water bodies (present in two images with part of a river) and other objects that look very different from everything else (e.g., containers, tennis courts, swimming pools), which are usually not relevant in semantic objects classification in urban scenes.

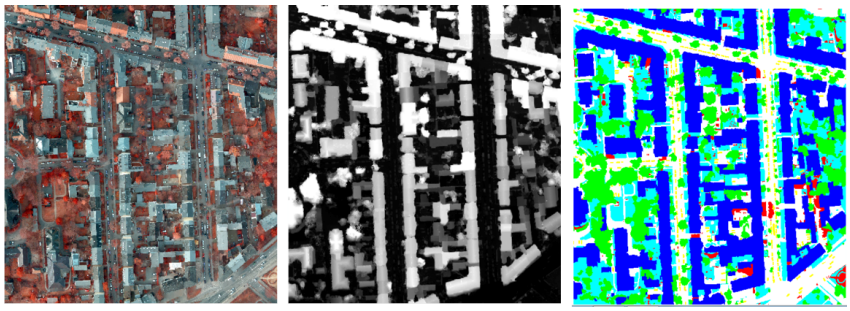


Figure 4.2. Example patches of the semantic object classification contest with (a) true orthophoto, (b) DSM, and (c) ground truth

Data collection method

The sampling distance of the TOP on the ground is 5 cm. *Trimble INPHO OrthoVista* software was used to generate the TOP mosaic. To avoid a lack of data areas ("holes") in the TOP, the authors have selected the patches from the central part of the TOP mosaic and none at the edges. The remaining small holes in the TOP have been interpolated.

4.1.2 Data Usage

Given the large size of the images, a further subdivision of the images was adopted to process every single segment into which the map of the city of Potsdam was subdivided.

Every macroblock has been divided into non-overlapping crops of size 256×256 pixels. Since the images have dimensions of 6000×6000 pixels, to not define mini crops with padding borders, the excess images have been removed from the dataset that we have processed.

Not all macro-areas have the same density of structures and relevant elements. When we define the mini crops, some have a total uniformity concerning the elements of interest that we aim to classify. These last images that show homogeneity in the labeling are not valuable for us and could cause overfitting phenomena in the training phase. So to avoid this problem the areas 2_10, 2_11, 2_12, 2_13, 2_14, 3_10, 6_7, 6_13, 7_13, 7_9 have been removed.

By eliminating the areas mentioned above twenty-eight macroblocks remained, of which twenty-two were employed to compose the training set while the remaining six were used as validation sets.

Pre-Training Processing

For pretraining, we performed data augmentation according to the self-distillation technique adopted in the paper "Emerging Properties in Self-Supervised Vision Transformers" [6]. For this phase, the data used relate exclusively to the high-resolution crops treue orthophoto (TOP) with a size of 256×256 pixels extracted from the 22 areas contained in the training set defined by us.

As previously discussed, each image is divided into sub-crops during this training phase. These crops will consist of global crops of size 224×224, processed by the teacher network, and local crops of 96x96 pixels to feed the student network.

A random cut of the original image is performed to extract the global crops. The cut is between the dimensions of 14% and 100% of the original image and then scaled to 224×224 pixels utilizing bicubic interpolation. In contrast, a random cut of the original image is performed between the image's 5% and 40% dimensions to extract the local crop and then scaled to 96×96 pixels through bicubic interpolation.

Each crop is subjected to a transformation pipeline that provides for random flipping with a probability of 50%, random color jittering with parameters brightness $= 0.4$, contrast $= 0.4$, saturation $= 0.2$, hue $= 0.1$ with a probability of 80%, and random gray scales with a probability of 20%. Finally, each crop is normalized with parameters $(0.5595, 0.6040, 0.5675)$ For μ, $(0.5608, 0.5995, 0.5710)$ For σ for R, G, B values.

Before passing through the normalization operation to a global crop, a Gaussian Gaussian noise filter is applied with parameter σ equivalent to 1.0. While the second is applied to a Gaussian noise filter with parameter σ equivalent to 0.1 and a solarization filter with a threshold of 0.2.

On the other hand, as regards the local-crops, before being normalized, a Gaussian noise filter with a causal value of σ between $0.1 - 2.0$ is applied with a probability of 50%.

Image Synthesis Processing

In the process to train the MAResUNet part of the network no data augmentation has been performed over the inputs images. The only transformation applied was a normalization operation.

4.2 Training Information

4.2.1 Loss Functions

Cross Entropy Loss

In order to compute and back propagate the error to update the networks' weights we use two different loss functions. If the model does not use the vector-quantization we use the classical and widely adopted cross entropy loss. The function is defined in the following way for discrete probability distributions:

$$L = H(p, q) = -E_p[log(q)] = H(p) + DL(p||q) = -\sum_{x \in X} p(x) \log q(x)$$

Where the term $DL(p||q)$ is the *Kullback–Leibler divergence* of p from q (also known as the relative entropy of p with respect to q).

VQ-VAE Loss

Instead, for the models that employ the vector-quantization codebook layer, the loss function that was adopted is a the one reported in the paper of VQ-VAE [38].

$$L = \log(p(x|q(x))) + ||\text{sg}[z_e(x)] - e||_2^2 + \beta||z_e(x) - \text{sg}[e]||_2^2$$

The first term is the reconstruction loss which optimizes the decoder and the encoder. The VQ objective uses the l2 error to move the embedding vectors e_i towards the encoder outputs $z_e(x)$ as shown in the equation above. Finally, since the volume of the embedding space is dimensionless, it can grow arbitrarily if the embeddings e_i do not train as fast as the encoder parameters. To ensure the encoder commits to an embedding and its output does not grow, we add a commitment loss, represented by the third term loss equation. The *sg* stands for the stop-gradient operator defined as identity at forward computation time and has zero partial derivatives, thus effectively constraining its operand to be a non-updated constant. The decoder optimizes the first loss term only, the encoder optimizes the first and the last loss terms, and the middle loss term optimizes the embeddings. Since we assume a uniform prior for z, the KL term that usually appears in the ELBO is constant *w.r.t.* The encoder parameters can thus be ignored for training.

DINO Loss

The DINO loss was used to train the backbones through self-supervised learning. This loss function refers to the article *"Emerging Properties in Self-Supervised Vision Transformers"* [7] and is structured as follows. As we have already mentioned in OBoW for pre-training two sets of images are generated through various image augmentation processes. This set contains two global view x_1^g, x_2^g and several local views of smaller resolution. All crops are passed through the student while only the global views are passed through the teacher, therefore encouraging "local-to-global" correspondences.

$$L = \sum_{x \in x_1^g, x_2^g} \sum_{x' \in V, x' \neq x} H(P_t(x), P_s(x^{'})) = \sum_{x \in x_1^g, x_2^g} \sum_{x' \in V, x' \neq x} -P_t(x) \log P_s(x^{'})$$

4.2.2 Evaluation Metrics

mIoU

The Intersection over Union (IoU) metric, also referred to as the *Jaccard index*, is a method to quantify the percent overlap between the target mask and our prediction output. This metric is closely related to the Dice coefficient which is often used as a loss function during training.

the IoU metric measures the number of pixels common between the target and prediction masks divided by the total number of pixels present across both masks.

$$IoU = \frac{target \cap prediction}{target \cup prediction}$$

Pixel Accuracy

An alternative metric to evaluate a semantic segmentation is to report the percent of pixels in the image which were correctly classified.

When considering the per-class pixel accuracy, we are essentially evaluating a binary mask; a true positive represents a pixel that is correctly predicted to belong to the given class (according to the target mask), whereas a true negative represents a correctly identified as not belonging to the given class.

$$accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

4.2.3 Training Details

All models have been trained for 100 epochs, beyond which they have all reached the state of convergence. The pre-training of the networks having a backbone function was brought according to the parameters below, following the implementation of the original article, using the dataset defined in the previous section regarding data

processing as a training set. Learning rate schedulers were used to optimize times and reduce the related computational loads. Regarding the self distillation phase, a cosine scheduler was used, while a StepLR scheduler was used for the fine-tuning phase of the networks having the function of semantic segmentation. All the details are reported in tables [**4.1**] and [**4.2**].

backbone	**scheduler**	**weight decay**	τ_s	τ_t	**epochs**	**initial learning rate**
ResNet18	cosine scheduler	0.04 to 0.4	0.1	0.04 to 0.07	100	0.0005 x batch_size/256
ResNet34	cosine scheduler	0.04 to 0.4	0.1	0.04 to 0.07	100	0.0005 x batch_size/256

Table 4.1. DINO training information, τ_s : temperature student, τ_t : temperature teacher

Model	**backbone**	**pretrain**	**optimizer**	**scheduler**				**epochs**	**batch_size**	**loss**
				type	**step_size**	**gamma**	**initial learning rate**			
MAResUNet	ResNet18	ImageNet	Adam	StepLR	15	0.4	(1e-3)	100	32	CE
MAResUNet	ResNet34	ImageNet	Adam	StepLR	15	0.4	(1e-3)	100	32	CE
MAResUNet	ResNet18	DINO	Adam	StepLR	15	0.4	(1e-3)	100	32	CE
MAResUNet	ResNet34	DINO	Adam	StepLR	15	0.4	(1e-3)	100	32	CE
MAResUNet-VQ	ResNet18	ImageNet	Adam	StepLR	15	0.4	(1e-3)	100	32	MSE+VQ_LOSS
MAResUNet-VQ	ResNet34	ImageNet	Adam	StepLR	15	0.4	(1e-3)	100	32	MSE+VQ_LOSS
MAResUNet-VQ	ResNet18	DINO	Adam	StepLR	15	0.4	(1e-3)	100	32	MSE+VQ_LOSS
MAResUNet-VQ	ResNet34	DINO	Adam	StepLR	15	0.4	(1e-3)	100	32	MSE+VQ_LOSS

Table 4.2. Training information for each model. CE : Cross Entropy Loss, MSE : Mean Square Error Loss, VQ_Loss : vector qauntizer loss.

4.3 Results

4.3.1 Test Scores

Model	backbone	pretrain	Accuracy per Class					
			Background	ImSurf	Car	Building	LowVeg	Tree
MAResUNet	ResNet18	ImageNet	**0.207613**	0.881230	**0.837966**	0.895138	**0.807360**	0.733300
MAResUNet	ResNet34	ImageNet	0.204663	**0.890064**	0.831177	0.894049	0.783754	**0.754808**
MAResUNet	ResNet18	DINO	0.114783	0.866064	0.766446	0.882792	0.673407	0.669019
MAResUNet	ResNet34	DINO	0.116300	0.858134	0.780328	0.887700	0.632859	0.705604
MAResUNet-VQ	ResNet18	ImageNet	0.108320	0.859215	0.797041	0.860000	0.725620	0.712245
MAResUNet-VQ	ResNet34	ImageNet	0.103615	0.867067	0.829848	0.872386	0.652675	0.723041
MAResUNet-VQ	ResNet18	DINO	0.035967	0.851734	0.732345	0.879178	0.570784	0.601483
MAResUNet-VQ	ResNet34	DINO	0.036300	0.843620	0.763357	**0.902357**	0.548373	0.609042

Table 4.3. Pixel accuracy scores from test phase.

Model	backbone	pretrain	mIoU per Class					
			Background	ImSurf	Car	Building	LowVeg	Tree
MAResUNet	ResNet18	ImageNet	0.148977	**0.716510**	**0.732305**	0.800466	**0.608436**	0.619789
MAResUNet	ResNet34	ImageNet	**0.150497**	0.708379	0.731492	**0.806703**	0.606029	**0.622545**
MAResUNet	ResNet18	DINO	0.092629	0.662431	0.658155	0.750052	0.515095	0.498773
MAResUNet	ResNet34	DINO	0.093808	0.660249	0.644279	0.748926	0.504074	0.499953
MAResUNet-VQ	ResNet18	ImageNet	0.091482	0.650338	0.669445	0.725899	0.558819	0.560490
MAResUNet-VQ	ResNet34	ImageNet	0.090156	0.653236	0.674301	0.746356	0.521225	0.518349
MAResUNet-VQ	ResNet18	DINO	0.034766	0.612845	0.562405	0.709556	0.443394	0.425335
MAResUNet-VQ	ResNet34	DINO	0.035200	0.622509	0.577945	0.699901	0.434970	0.430464

Table 4.4. mIoU scores from test phase.

Model	backbone	pretrain	OA	F1	mIoU
MAResUNet	ResNet18	ImageNet	0.790582	0.**721673**	0.**604414**
MAResUNet	ResNet34	ImageNet	**0.791249**	0.721502	0.604274
MAResUNet	ResNet18	DINO	0.741404	0.656554	0.529523
MAResUNet	ResNet34	DINO	0.739260	0.652705	0.525215
MAResUNet-VQ	ResNet18	ImageNet	0.749910	0.668361	0.542746
MAResUNet-VQ	ResNet34	ImageNet	0.743700	0.659500	0.533937
MAResUNet-VQ	ResNet18	DINO	0.702228	0.592809	0.464717
MAResUNet-VQ	ResNet34	DINO	0.703382	0.594966	0.466832

Table 4.5. Scores from test phase.

4.3.2 Validation Scores

Model	backbone	pretrain	Accuracy per Class					
			Background	ImSurf	Car	Building	LowVeg	Tree
MAResUNet	ResNet18	ImageNet	**0.963921**	**0.989986**	**0.953666**	**0.996563**	**0.984027**	0.985651
MAResUNet	ResNet34	ImageNet	0.953774	0.989382	0.952051	0.996358	0.981020	**0.986299**
MAResUNet	ResNet18	DINO	0.421115	0.935828	0.892736	0.987469	0.841021	0.905704
MAResUNet	ResNet34	DINO	0.416618	0.923470	0.885593	0.986304	0.817395	0.822869
MAResUNet-VQ	ResNet18	ImageNet	0.417500	0.939221	0.879882	0.983019	0.845985	0.845949
MAResUNet-VQ	ResNet34	ImageNet	0.374469	0.935931	0.891765	0.980780	0.822525	0.781699
MAResUNet-VQ	ResNet18	DINO	0.171728	0.887304	0.825941	0.951475	0.738283	0.581014
MAResUNet-VQ	ResNet34	DINO	0.178181	0.884719	0.831619	0.956257	0.727274	0.605574

Table 4.6. Pixel accuracy scores from validation phase.

Model	backbone	pretrain	mIoU per Class					
			Background	ImSurf	Car	Building	LowVeg	Tree
MAResUNet	ResNet18	ImageNet	**0.922911**	**0.979468**	**0.919333**	**0.994083**	**0.965180**	**0.974108**
MAResUNet	ResNet34	ImageNet	0.919658	0.977756	0.916698	0.993909	0.960782	0.970244
MAResUNet	ResNet18	DINO	0.369162	0.862386	0.821021	0.968801	0.716659	0.833712
MAResUNet	ResNet34	DINO	0.360288	0.846912	0.794362	0.964099	0.666785	0.729376
MAResUNet-VQ	ResNet18	ImageNet	0.371715	0.870175	0.803154	0.956418	0.711559	0.761481
MAResUNet-VQ	ResNet34	ImageNet	0.330136	0.855976	0.802528	0.949822	0.684872	0.689373
MAResUNet-VQ	ResNet18	DINO	0.167055	0.768959	0.668802	0.894876	0.534041	0.466240
MAResUNet-VQ	ResNet34	DINO	0.171959	0.769295	0.668120	0.891223	0.551043	0.490592

Table 4.7. mIoU scores from validation phase.

Model	backbone	pretrain	OA	F1	mIoU
MAResUNet	ResNet18	ImageNet	**0.990642**	**0.97890**6	**0.959180**
MAResUNet	ResNet34	ImageNet	0.989849	0.977495	0.956508
MAResUNet	ResNet18	DINO	0.934129	0.844922	0.761957
MAResUNet	ResNet34	DINO	0.920964	0.821984	0.726970
MAResUNet-VQ	ResNet18	ImageNet	0.928538	0.835005	0.745750
MAResUNet-VQ	ResNet34	ImageNet	0.918518	0.812876	0.718784
MAResUNet-VQ	ResNet18	DINO	0.857786	0.695221	0.583329
MAResUNet-VQ	ResNet34	DINO	0.860873	0.700642	0.590372

Table 4.8. Scores from validation phase.

4.3.3 Discussion

First of all, the relative performance of the models can be discussed based on tables [**4.5**] and [**4.8**], which report the accuracy, mIoU, and F1 scores. It can be readily noticed how the models that do not use vector quantization or pre-training techniques through self distillation based on the DINO method stand out. From these results, it can also be deduced that among the models that use ResNet18 and ResNet34 as the backbone, the one that manages to obtain better scores is the model with fewer parameters. Now, by comparing the various scores in the validation and test phases, we notice a sharp decline between these two different evaluation phases. We can say with enough confidence that the decrease is due to a phenomenon of overfitting of the various models. Although all the models are affected, we note a certain homogeneity, which could be due to a more significant weight from the data distribution of the dataset than from the architecture of the model itself or the training methodology followed. Remember that the target classes have large variation within the same class and low variation between different classes.

To go into further detail, we can address the data in tables [**4.4, 4.5, 4.7, 4.8**], which report the various scores on the metrics of pixel accuracy, and mean intersection over union (mIoU more commonly known as the Jaccard index), for the individual segmentation classes of interest. For each model, the class that introduces the greatest amount of noise is the class related to the background. This class, which represents and helps classify areas such as swimming pools, sports fields, and car parks, has the most heterogeneity and introduces the most significant number of false positives in the inference phase.

It introduces numerous classification errors, considerably lowers the average accuracy and global mIoU value of the individual models. It is also evident from the extracted samples shown below in figures [**4.3, 4.4**] that only the models pre-trained with ImageNet and which do not use Vector Quantization can detect this class (represented by the red color). These assumptions are supported by the tests. In the presence of vector quantization or pre-training based on DINO, the various scores drop drastically, reaching around ten percent in precision in both methods.

Passing to the analysis of the remaining segmentation classes, we can mention how *"Buildings"* stands out among all, which manages to obtain the best metrics. The class of *"Car"* distances itself a little while always maintaining an excellent representation in the inference phase.

With regard to the classes that represent *"Tree"* and *"LowVeg"* (low vegetation), it was not surprising to find that the results were worse due to the difficulty of the model in establishing the contours of the segmentation masks deriving from these latter classes of interest. In fact, unlike cars and buildings, the vegetation does not have well-defined edges; moreover, it is widely heterogeneous both in colors and in expansion. It is often easy to confuse trees with low vegetation (as it could be a meadow or a flowery field) due to shadows generated by the passage of cloudy masses, mists, or shadows cast by tall buildings. It should also be

considered that noise introduces the background class, which turns out to be the one in greatest contact with the latter two. The images [**4.3, 4.4**] make it easy to highlight how the areas affected by a misclassification are the "Tree" and "LowVeg".

As a final, but not less important, remark the self distillation learning techniques proved to be ineffective compared with the transfer learning methodology, which lays its foundations on pre-training with ImageNet. We can hypothesize that the ImageNet pre-training has obtained better results due to the difference in the size of the datasets. While our training set from which the encoders were trained based on the DINO methodology had a number of about $11,000$ entries, ImageNet can boast a number of entries in the millions. If added to the overfitting phenomenon to which our models are subject, with an emphasis on tests that make use of the pre-training technology we use, this assumption translates into a small number of samples on which to base the training of our networks. Therefore, it is assumed that with a larger dataset, the results may improve until they reach and surpass those set by the best model available among those tested.

4.3.4 Outputs

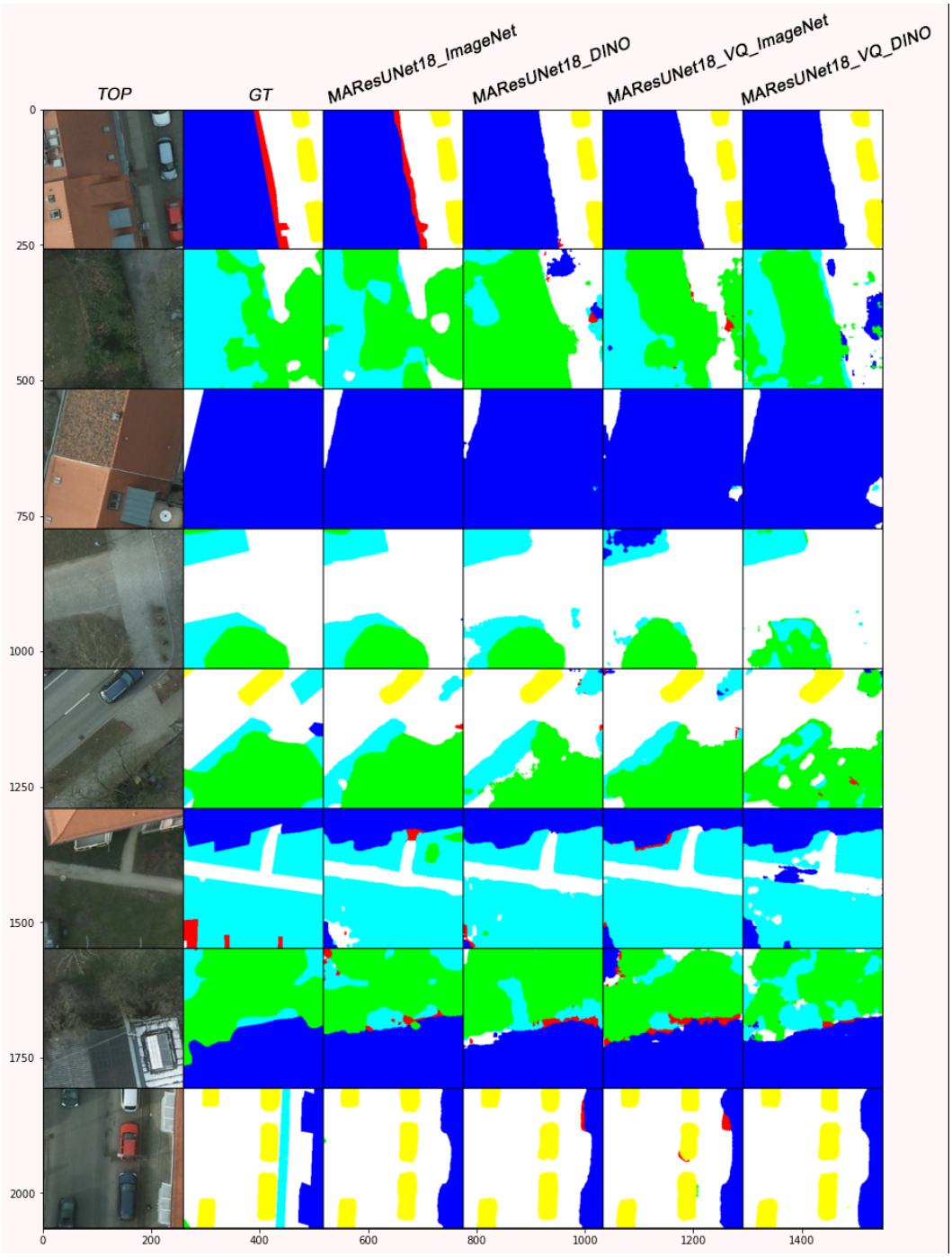


Figure 4.3. MAResUNet models using ResNet18 as the backbone. TOP : Treue Orthophoto, GT : Ground Truth (target).

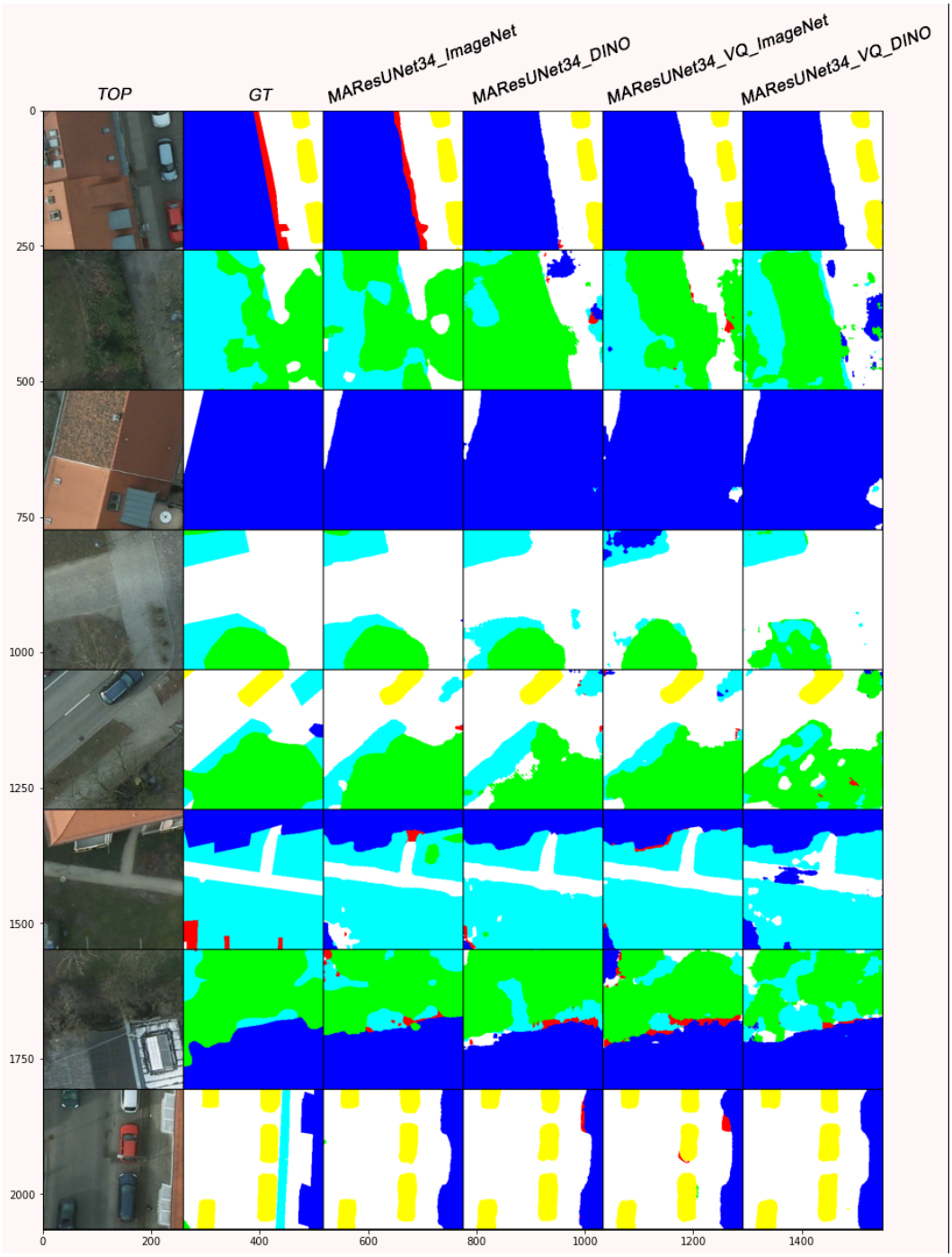


Figure 4.4. MAResUNet models using ResNet34 as the backbone. TOP : Treue Orthophoto, GT : Ground Truth (target).

Chapter 5

Conclusions and Perspectives

Starting from the work on Masocci et al. [33] by previous authors, the present study was directed at obtaining an improved version of the model, with special reference to the level of accuracy. In particular, the study aimed to improve the precision in rendering the predicted images with respect to the relevant target ones in semantic segmentation.

To this end, various experiments were conducted, illustrated in the previous chapters. All the performed tests' models share the same architecture, as described in the appropriate sections, and differ in their sizes, pre-training method, and the presence of quantization.

The models employed as encoders, used as the backbone for the framework treated in this thesis, are ResNet18 and ResNet34.

As highlighted in the previous discussion section, those models which relied on less parameters, i.e., those that used a ResNet18 as the backbone, turned to be the best.

In many cases, models with more parameters achieve higher scores at the expense of a higher computational cost, present both in the training and inference phases. This allows us to define good models that do not require excessive effort on the part of the machines to function. Powerful machines require a large amount of energy to operate while using models with fewer parameters, and it is possible to save in terms of energy, an issue that should not be overlooked, especially in our case where we are going to analyze images that can be processed in real-time by drones or satellites.

As far as future developments are concerned, one attractive possibility would be to devise new models capable of operating on machines with a lower energy demand. We have already seen how implementing an attention mechanism with linear cost, described in the paper by Li et al. [25], was able to reduce the complexity of the required calculations while maintaining the performance of the model.

Studies in the semantic classification of remote sensing images are increasingly

improving. New models are emerging over the years, increasing the accuracy of sensors and instruments in the industry.

Nowadays, transformers seem to be the basis of the "revolution" currently underway and the key to gaining access to increasingly sophisticated, precise, and efficient models.

Furthermore, new approaches are currently being developed to reduce the computational load deriving from the calculation of the attention mechanism.

Another most promising field for ongoing and future consideration is that relating to training techniques. The teacher-student paradigm, previously addressed in this thesis, is gaining credibility as a necessary standard to compete with the best models currently available in the literature.

With some due precautions, it may be possible to adapt these new methodologies to allow for their inclusion and utilization into the specific framework used in this thesis.

Bibliography

[1] Anil, R., Pereyra, G., Passos, A., Ormandi, R., Dahl, G. E., and Hinton, G. E. Large scale distributed neural network training through online distillation. *arXiv preprint arXiv:1804.03235*, (2018).

[2] Bank, D., Koenigstein, N., and Giryes, R. Autoencoders. *arXiv preprint arXiv:2003.05991*, (2020).

[3] Bento, C. Multilayer perceptron explained with a real-life example and python code: Sentiment analysis. www.towardsdatascience.com. Accessed: 2022-05-14.

[4] Bishop, C. M. and Nasrabadi, N. M. *Pattern recognition and machine learning*, vol. 4. Springer (2006).

[5] Caron, M., Misra, I., Mairal, J., Goyal, P., Bojanowski, P., and Joulin, A. Unsupervised learning of visual features by contrasting cluster assignments. *Advances in Neural Information Processing Systems*, **33** (2020), 9912.

[6] Caron, M., Touvron, H., Misra, I., Jégou, H., Mairal, J., Bojanowski, P., and Joulin, A. Emerging properties in self-supervised vision transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 9650–9660 (2021).

[7] Caron, M., Touvron, H., Misra, I., Jégou, H., Mairal, J., Bojanowski, P., and Joulin, A. Emerging properties in self-supervised vision transformers. *CoRR*, **abs/2104.14294** (2021). Available from: `https://arxiv.org/abs/2104.14294`, `arXiv:2104.14294`.

[8] Chen, L.-C., Papandreou, G., Kokkinos, I., Murphy, K., and Yuille, A. L. Deeplab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected crfs. *IEEE transactions on pattern analysis and machine intelligence*, **40** (2017), 834.

[9] Chen, T., Kornblith, S., Norouzi, M., and Hinton, G. A simple framework for contrastive learning of visual representations. In *International conference on machine learning*, pp. 1597–1607. PMLR (2020).

[10] Chen, X. and He, K. Exploring simple siamese representation learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 15750–15758 (2021).

[11] Cheng, G. and Han, J. A survey on object detection in optical remote sensing images. *ISPRS Journal of Photogrammetry and Remote Sensing*, **117** (2016), 11.

[12] Esser, P., Rombach, R., and Ommer, B. Taming transformers for high-resolution image synthesis. *CoRR*, **abs/2012.09841** (2020). Available from: `https://arxiv.org/abs/2012.09841`, `arXiv:2012.09841`.

[13] Farooq, Z., Zaman, T., Khan, M. A., Muyeen, S., Ibeas, A., et al. Artificial neural network based adaptive control of single phase dual active bridge with finite time disturbance compensation. *IEEE Access*, **7** (2019), 112229.

[14] Gidaris, S., Bursuc, A., Puy, G., Komodakis, N., Cord, M., and Perez, P. Obow: Online bag-of-visual-words generation for self-supervised learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 6830–6840 (2021).

[15] Gou, J., Yu, B., Maybank, S. J., and Tao, D. Knowledge distillation: A survey. *International Journal of Computer Vision*, **129** (2021), 1789.

[16] Grill, J., et al. Bootstrap your own latent: A new approach to self-supervised learning. *CoRR*, **abs/2006.07733** (2020). Available from: `https://arxiv.org/abs/2006.07733`, `arXiv:2006.07733`.

[17] He, K., Fan, H., Wu, Y., Xie, S., and Girshick, R. Momentum contrast for unsupervised visual representation learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 9729–9738 (2020).

[18] He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778 (2016).

[19] Higgins, I., Matthey, L., Pal, A., Burgess, C., Glorot, X., Botvinick, M., Mohamed, S., and Lerchner, A. beta-vae: Learning basic visual concepts with a constrained variational framework. (2016).

[20] Hinton, G., Vinyals, O., Dean, J., et al. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, **2** (2015).

[21] James Chen, R. E., Charles Potters. Exponential moving average (ema). www.investopedia.com. Accessed: 2022-05-14.

[22] Kingma, D. P. and Welling, M. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, (2013).

[23] Kirillov, A., Girshick, R., He, K., and Dollár, P. Panoptic feature pyramid networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 6399–6408 (2019).

[24] Li, C., Yang, J., Zhang, P., Gao, M., Xiao, B., Dai, X., Yuan, L., and Gao, J. Efficient self-supervised vision transformers for representation learning. *arXiv preprint arXiv:2106.09785*, (2021).

[25] Li, R., Su, J., Duan, C., and Zheng, S. Linear attention mechanism: An efficient attention for semantic segmentation. *arXiv preprint arXiv:2007.14902*, (2020).

[26] Li, R., Su, J., Duan, C., and Zheng, S. Linear attention mechanism: An efficient attention for semantic segmentation. *CoRR*, **abs/2007.14902** (2020). Available from: `https://arxiv.org/abs/2007.14902`, `arXiv:2007.14902`.

[27] Li, R., Zheng, S., Duan, C., Su, J., and Zhang, C. Multistage attention resu-net for semantic segmentation of fine-resolution remote sensing images. *IEEE Geoscience and Remote Sensing Letters*, **19** (2021), 1.

[28] Li, R., Zheng, S., Duan, C., Su, J., and Zhang, C. Multistage attention resu-net for semantic segmentation of fine-resolution remote sensing images. *IEEE Geoscience and Remote Sensing Letters*, **19** (2022), 1. `doi:10.1109/LGRS.2021.3063381`.

[29] Li, X., Chen, H., Qi, X., Dou, Q., Fu, C.-W., and Heng, P.-A. H-denseunet: hybrid densely connected unet for liver and tumor segmentation from ct volumes. *IEEE transactions on medical imaging*, **37** (2018), 2663.

[30] Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., Lin, S., and Guo, B. Swin transformer: Hierarchical vision transformer using shifted windows. *arXiv preprint arXiv:2103.14030*, (2021).

[31] Long, J., Shelhamer, E., and Darrell, T. Fully convolutional networks for semantic segmentation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 3431–3440 (2015).

[32] Lu, J., Feng, J., Fan, Z., Huang, L., Zheng, C., Li, W., et al. Automated strabismus detection based on deep neural networks for telemedicine application. *Knowledge-based systems*, **13** (2019).

[33] Marsocci, V., Scardapane, S., and Komodakis, N. Mare: Self-supervised multi-attention resu-net for semantic segmentation in remote sensing. *Remote Sensing*, **13** (2021), 3275.

[34] Razavi, A., van den Oord, A., and Vinyals, O. Generating diverse high-fidelity images with vq-vae-2. In *Advances in neural information processing systems*, pp. 14866–14876 (2019).

[35] Ronneberger, O., Fischer, P., and Brox, T. U-net: Convolutional networks for biomedical image segmentation. In *International Conference on Medical image computing and computer-assisted intervention*, pp. 234–241. Springer (2015).

[36] Touvron, H., Cord, M., Douze, M., Massa, F., Sablayrolles, A., and Jégou, H. Training data-efficient image transformers & distillation through attention. *CoRR*, **abs/2012.12877** (2020). Available from: `https://arxiv.org/abs/2012.12877`, `arXiv:2012.12877`.

[37] Touvron, H., Cord, M., Douze, M., Massa, F., Sablayrolles, A., and Jégou, H. Training data-efficient image transformers & distillation through attention. In *International Conference on Machine Learning*, pp. 10347–10357. PMLR (2021).

[38] van den Oord, A., Vinyals, O., and Kavukcuoglu, K. Neural discrete representation learning. *CoRR*, **abs/1711.00937** (2017). Available from: `http://arxiv.org/abs/1711.00937`, `arXiv:1711.00937`.

[39] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. *Advances in neural information processing systems*, **30** (2017).

[40] Wikipedia contributors. Variational autoencoder — Wikipedia, the free encyclopedia. `https://en.wikipedia.org/w/index.php?title=Variational_autoencoder&oldid=1087184794` (2022). [Online; accessed 18-May-2022].

[41] Zhang, Y., Xiang, T., Hospedales, T. M., and Lu, H. Deep mutual learning. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 4320–4328 (2018).